

IPOG — Instituto de Pós-Graduação e Graduação

Projeto Integrador • Área 03 — Banco de Dados

Modelagem e análise de dados de vendas

Northwind Traders • PostgreSQL e MongoDB

Entrega 4 — Relatório final

Do dado ao critério de escolha: modelagem, análises comparativas e guia de decisão

Giovanne Espindola

Trabalho individual

Outubro de 2026

Sumário

Seção	Página
1. Introdução e objetivo	3
2. Compreensão do negócio	4
3. Modelo conceitual	5
4. Análise exploratória e qualidade dos dados	7
5. Arquitetura híbrida	12
6. Modelo relacional	13
7. Modelo documental	15
8. Transformação, carga e validação dos documentos	17
9. Índices no MongoDB	19
10. Conferência da migração	20
11. Regras comuns às análises	21
12. Views e procedures	22
13. Consultas analíticas e resultados	23
14. Recursos próprios do MongoDB e MapReduce	32
15. Comparação de sintaxe e complexidade	33
16. Performance e otimização	35
17. Guia de decisão: quando usar cada tecnologia	38
18. Escalabilidade e manutenção	39
19. Conclusões	40
20. Como reproduzir	41
Referências	42
Apêndice A — Consultas SQL e pipelines completos	43
Apêndice B — Views, procedures e recursos documentais	64

1. Introdução e objetivo

A Northwind Traders é uma distribuidora fictícia de alimentos, usada como base didática de vendas. Seus registros cobrem clientes, produtos, fornecedores, pedidos e a equipe comercial. Este projeto parte de uma pergunta simples: **qual é a ferramenta certa para cada trabalho com esses dados?** Para respondê-la, o mesmo conjunto foi implementado em um banco relacional (PostgreSQL) e em um banco orientado a documentos (MongoDB), e as mesmas 16 perguntas de negócio foram respondidas nos dois.

O objetivo de negócio é entender como o valor dos pedidos se distribui entre produtos, categorias, clientes, vendedores e meses, além de observar o estoque e os envios. O objetivo técnico é comparar os dois modelos em correção dos resultados, forma de escrever as consultas, desempenho e manutenção, e transformar essa comparação em critérios de escolha.

1.1 Método: CRISP-DM

O CRISP-DM organiza projetos de dados em seis fases [1]. Aqui ele foi adaptado a um projeto de banco de dados: a “modelagem” é das estruturas de armazenamento, não de modelos estatísticos.

Fase	O que foi feito
Compreensão do negócio	Contexto da empresa, perguntas a responder e limites da base.
Compreensão dos dados	Inventário, nulos, chaves, faixas de valores e precisão decimal.
Preparação dos dados	Schema relacional próprio e transformação das tabelas em documentos.
Modelagem	Modelo relacional normalizado e modelo documental com itens incorporados.
Avaliação	Comparação dos resultados nos dois bancos e medição de desempenho.
Implantação	Guia de decisão, scripts reproduzíveis e documentação.

As fases se retroalimentaram. A falta de custos na base, descoberta na exploração, fez com que as perguntas financeiras tratassem do valor dos pedidos, e não de lucro.

1.2 Organização deste relatório

O documento é cumulativo e segue a ordem das semanas do projeto [3].

Entrega	Semanas	Capítulos
1	02 — compreensão do negócio e dos dados	1 a 4
2	03 e 04 — modelagem relacional e NoSQL	5 a 10
3	05 e 06 — consultas, views, procedures, pipelines e performance	11 a 16
4	07 — guia de decisão, escalabilidade, conclusões e reprodução	17 a 20

Os códigos completos das 16 consultas SQL e dos 16 pipelines estão no Apêndice A. As views, as procedures, as demonstrações de recursos do MongoDB e o MapReduce estão no Apêndice B.

1.3 Base e ferramentas

A base é a adaptação do Northwind para PostgreSQL mantida no repositório **pthom/northwind_psql** [2]. Os dois bancos rodam em containers Docker: PostgreSQL 16 e MongoDB 7. A exploração usou SQL e um notebook Jupyter; a transformação para documentos foi feita em SQL e carregada com o mongosh; a validação e as medições usaram Python.

2. Compreensão do negócio

A empresa vende produtos de várias categorias, comprados de fornecedores cadastrados. Os clientes fazem pedidos, registrados por funcionários. Cada pedido tem um ou mais itens, com produto, quantidade, preço praticado e desconto, e indica uma transportadora. Os funcionários formam uma hierarquia e atuam em territórios agrupados em regiões.

A base cobre o pedido até o envio. Ela não registra pagamento, entrega ao cliente, devoluções nem custo dos produtos. Isso define o que o projeto pode e o que não pode afirmar.

2.1 Perguntas de negócio

Pergunta	Para que serve
Quais categorias e produtos concentram o valor dos pedidos?	Saber de onde vem a receita registrada.
Como variam o volume e o valor médio dos pedidos ao longo dos meses?	Acompanhar a evolução comercial.
Quais clientes compram mais e quais pararam de comprar?	Orientar ações de relacionamento.
Quanto foi registrado por vendedor e por equipe?	Descrever a distribuição das vendas.
Quais produtos estão abaixo do ponto de reposição?	Sinalizar riscos no estoque cadastrado.
Quanto tempo leva do pedido ao envio, por transportadora?	Observar a operação de envio.

Essas seis perguntas foram desdobradas nas 16 análises do capítulo 13.

2.2 Definições usadas em todo o relatório

Valor do pedido: soma de preço praticado $\times$ quantidade $\times$ (1 – desconto) de seus itens, sem frete. É o valor registrado, não o valor recebido. A base não identifica a moeda, então os valores aparecem em unidades monetárias.

Ticket médio: valor total dividido pelo número de pedidos. Os itens são somados por pedido antes da média, para que um pedido com cinco itens conte uma vez, e não cinco.

Recência: dias desde a última compra, contados a partir de 06/05/1998, data do último pedido da base. Usar a data de hoje tornaria todos os clientes igualmente antigos.

O que fica de fora: sem custos, não há margem nem lucro; sem pagamentos, não há receita recebida; sem data de chegada, não há prazo de entrega. As tabelas demográficas estão vazias, então a segmentação de clientes usa apenas o histórico de compras.

3. Modelo conceitual

O diagrama mostra as 11 entidades usadas no projeto e como elas se relacionam, sem tipos nem detalhes de armazenamento. ITEM DO PEDIDO e ATUAÇÃO são entidades associativas: a primeira liga pedido e produto; a segunda liga funcionário e território.

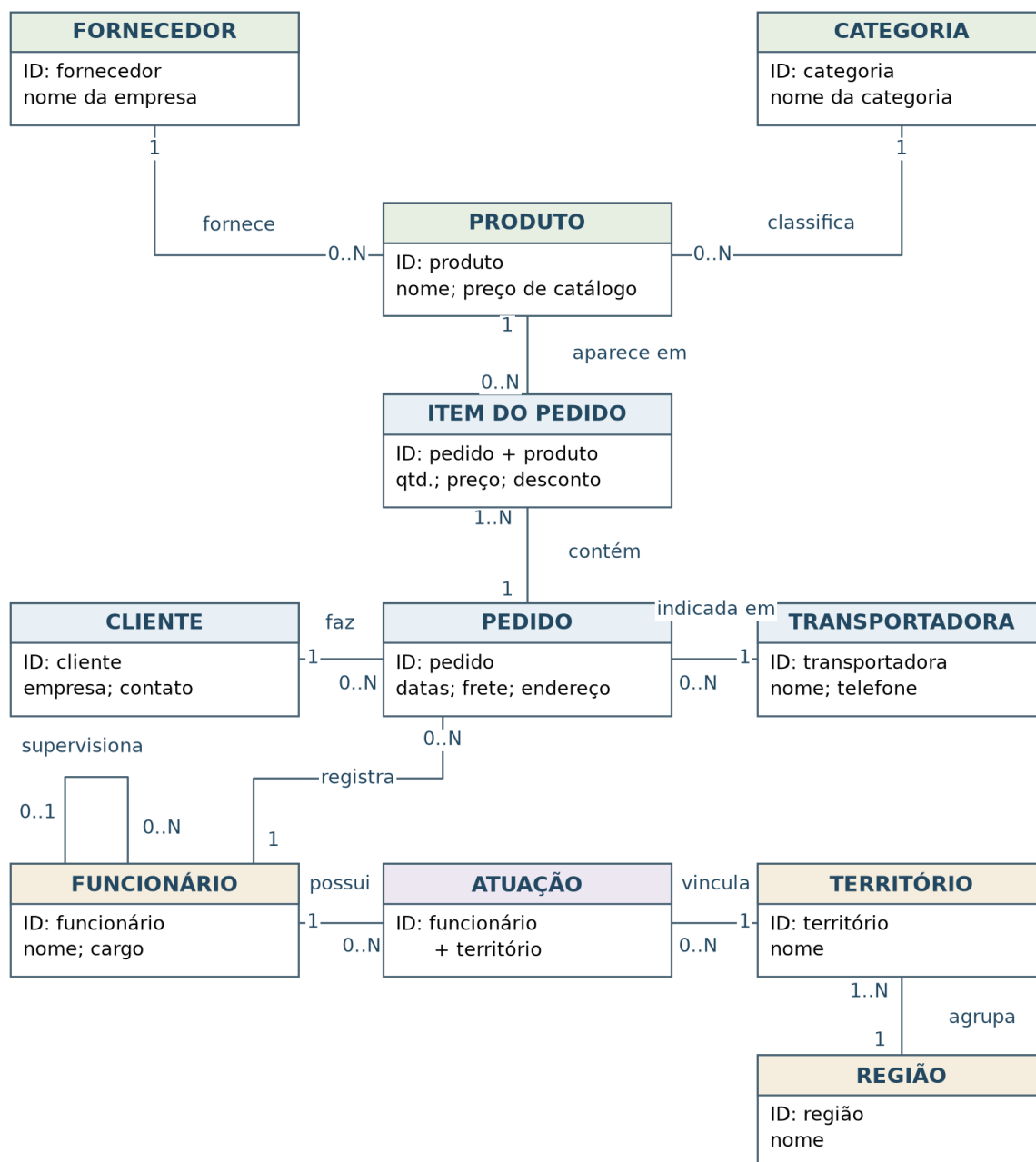


Figura 1 — Entidades e relacionamentos do recorte Northwind, elaborado a partir da base [2]. Os números indicam quantas ocorrências de um lado se relacionam com uma do outro: 1 = exatamente uma; 0..1 = opcional; 0..N = nenhuma ou várias; 1..N = uma ou várias.

3. Modelo conceitual — regras adotadas

As regras abaixo são decisões do projeto, compatíveis com os dados. Um caso que não aparece na amostra não foi tratado como proibido.

Relacionamento	Regra
Cliente e pedido	Cada pedido tem um cliente; um cliente pode ter zero ou vários pedidos.
Funcionário e pedido	Cada pedido tem um funcionário; um funcionário pode não ter pedidos.
Transportadora e pedido	Cada pedido indica uma transportadora; ela pode não ter pedidos.
Pedido e item	Um pedido tem um ou vários itens; cada item pertence a um pedido.
Produto e item	Cada item se refere a um produto; um produto pode não ter sido vendido.
Categoria e produto	Cada produto pertence a uma categoria.
Fornecedor e produto	Cada produto tem um fornecedor.
Região e território	Cada território pertence a uma região.
Funcionário e território	Relação N:N, registrada pela atuação.
Supervisão	Cada funcionário tem no máximo um superior e pode supervisionar vários.

ITEM DO PEDIDO é identificado pelo par pedido e produto: o mesmo produto não se repete num pedido, e a quantidade registra quantas unidades foram vendidas. Na amostra, cada um dos 49 territórios com atuação tem um único responsável, mas a relação N:N foi mantida porque o domínio permite mais de um.

A base também tem dois clientes sem pedidos, três transportadoras sem pedidos e quatro territórios sem funcionário. Esses registros foram preservados, porque “não comprou” é uma informação útil. As tabelas demográficas, vazias, e `us_states`, que não se liga ao restante, ficaram fora do recorte.

4. Análise exploratória e qualidade dos dados

4.1 Inventário

A exploração foi feita no schema **public**, que guarda a base original intocada, por consultas SQL e pelo notebook de perfilamento. São 14 tabelas e 92 colunas.

Tabela	Linhas	Colunas
categories	8	4
customer_customer_demo	0	2
customer_demographics	0	2
customers	91	11
employee_territories	49	2
employees	9	18
order_details	2155	5
orders	830	14
products	77	10
region	4	2
shippers	6	3
suppliers	29	12
territories	53	3
us_states	51	4

As 3.362 linhas misturam cadastros, pedidos e itens; o número que interessa ao negócio é outro: **830 pedidos e 2.155 itens**, em média 2,6 produtos por pedido.

Consulta 1 — Período e clientes com pedido.

```
SELECT min(order_date) AS primeiro_pedido,
       max(order_date) AS ultimo_pedido,
       count(*) AS pedidos,
       count(DISTINCT customer_id) AS clientes_com_pedido
FROM public.orders;
```

primeiro_pedido	ultimo_pedido	pedidos	clientes_com_pedido
1996-07-04	1998-05-06	830	89

Os pedidos vão de 04/07/1996 a 06/05/1998. Dos 91 clientes cadastrados, 89 compraram nesse período.

4. Análise exploratória — valores e concentração

4.2 Valor dos pedidos

Consulta 2 — Total e distribuição por pedido. Os valores são convertidos para decimal antes da multiplicação e arredondados só no resultado.

```
WITH totais AS (
  SELECT order_id,
         sum(unit_price::numeric * quantity *
              (1 - discount::numeric)) AS valor
  FROM public.order_details GROUP BY order_id
)
SELECT round(sum(valor), 2) AS total,
       round(avg(valor), 2) AS media,
       round(percentile_cont(0.5) WITHIN GROUP
              (ORDER BY valor)::numeric, 2) AS mediana,
       round(min(valor), 2) AS minimo,
       round(max(valor), 2) AS maximo
FROM totais;
```

total	media	mediana	minimo	maximo
1.265.793,04	1.525,05	943,25	12,50	16.387,50

O total registrado é **1.265.793,04**. O ticket médio (1.525,05) fica bem acima da mediana (943,25), porque alguns pedidos grandes, como o de 16.387,50, puxam a média para cima. Para descrever o pedido típico, a mediana é a medida mais fiel.

4.3 Participação das categorias

Consulta 3 — Valor por categoria.

```
SELECT c.category_name AS categoria,
       round(sum(d.unit_price::numeric * d.quantity *
                 (1 - d.discount::numeric)), 2) AS valor
FROM public.order_details d
JOIN public.products p ON p.product_id = d.product_id
JOIN public.categories c ON c.category_id = p.category_id
GROUP BY c.category_name ORDER BY valor DESC;
```

categoria	valor
Beverages	267.868,18
Dairy Products	234.507,29
Confections	167.357,23
Meat/Poultry	163.022,36
Seafood	131.261,74
Condiments	106.047,09
Produce	99.984,58
Grains/Cereals	95.744,59

Beverages e Dairy Products somam 502.375,47, cerca de **40% do valor**. É concentração de receita registrada, não de lucro, porque a base não tem custos.

4. Análise exploratória — período e dados ausentes

4.4 Distribuição no tempo

ano	pedidos	valor
1996	152	208.083,97
1997	408	617.085,20
1998	270	440.623,87

1996 e 1998 são anos incompletos: os registros começam em julho de 1996 e terminam em maio de 1998. Comparar os totais anuais daria a falsa impressão de que 1997 foi um ano excepcional. Por isso, as análises do capítulo 13 trabalham com meses e trimestres e marcam os períodos parciais.

4.5 Completude

Onze das 92 colunas têm algum nulo, quase todas de contato e endereço.

Coluna	Nulos	Total	%
customers.region	60	91	65,93
customers.postal_code	1	91	1,10
customers.fax	22	91	24,18
employees.region	4	9	44,44
employees.reports_to	1	9	11,11
orders.shipped_date	21	830	2,53
orders.ship_region	507	830	61,08
orders.ship_postal_code	19	830	2,29
suppliers.region	20	29	68,97
suppliers.fax	16	29	55,17
suppliers.homepage	24	29	82,76

Nenhum nulo foi preenchido com valor inventado. O nulo em employees.reports_to tem significado próprio: é o funcionário no topo da hierarquia.

Consulta 4 — Data de envio e tempo até o envio.

```
SELECT count(*) AS pedidos,
       count(*) FILTER (WHERE shipped_date IS NULL) AS sem_data,
       round(100.0 * count(*) FILTER
         (WHERE shipped_date IS NULL) / count(*), 2) AS pct_sem_data,
       round(avg(shipped_date - order_date), 2) AS dias_ate_envio
FROM public.orders;
```

pedidos	sem_data	pct_sem_data	dias_ate_envio
830	21	2,53	8,49

21 pedidos (2,53%) não têm data de envio. A média de 8,49 dias considera só os 809 pedidos com data. A ausência não prova que o pedido não saiu, e nenhum desses números mede a entrega ao cliente.

4. Análise exploratória — integridade e validade

4.6 Chaves e relacionamentos

A base tem 14 chaves primárias e 13 chaves estrangeiras, sem identificadores duplicados nem referências órfãs.

Consulta 5 — Verificação de relacionamentos.

```
SELECT 'pedido sem cliente válido' AS verificacao, count(*) AS casos
FROM public.orders o WHERE NOT EXISTS (
  SELECT 1 FROM public.customers c WHERE c.customer_id = o.customer_id)
UNION ALL
SELECT 'item sem pedido válido', count(*)
FROM public.order_details d WHERE NOT EXISTS (
  SELECT 1 FROM public.orders o WHERE o.order_id = d.order_id)
UNION ALL
SELECT 'pedido sem item', count(*)
FROM public.orders o WHERE NOT EXISTS (
  SELECT 1 FROM public.order_details d WHERE d.order_id = o.order_id);
```

verificacao	casos
pedido sem cliente válido	0
item sem pedido válido	0
pedido sem item	0

Todos os pedidos têm itens, então o ticket médio pode usar os 830 pedidos sem exclusões.

4.7 Faixas de valores

Consulta 6 — Valores fora das faixas e envio antes do pedido.

```
SELECT 'quantidade <= 0' AS verificacao, count(*) AS casos
FROM public.order_details WHERE quantity <= 0
UNION ALL
SELECT 'preço < 0', count(*) FROM public.order_details WHERE unit_price < 0
UNION ALL
SELECT 'desconto fora de [0, 1]', count(*)
FROM public.order_details WHERE discount < 0 OR discount >= 1
UNION ALL
SELECT 'envio anterior ao pedido', count(*)
FROM public.orders WHERE shipped_date < order_date;
```

verificacao	casos
quantidade <= 0	0
preço < 0	0
desconto fora de [0, 1)	0
envio anterior ao pedido	0

Preços vão de 2,00 a 263,50, quantidades de 1 a 130 e descontos de 0% a 25%, sem casos inválidos. A base original, porém, não tem nenhuma restrição CHECK: os dados estão corretos hoje, mas nada impede um valor inválido amanhã. O modelo relacional do projeto acrescenta essas regras (capítulo 6).

4. Análise exploratória — precisão e histórico

4.8 Decimais e momento do arredondamento

Na base original, preço e desconto são do tipo **real**, que guarda aproximações. Para dinheiro, o PostgreSQL recomenda **numeric** [4]. Converter evita novos erros, embora não recupere uma precisão que a origem já tenha perdido.

Também importa *quando* se arredonda.

Consulta 7 — Arredondar a soma ou arredondar cada item.

```
SELECT round(sum(unit_price::numeric * quantity *
                (1 - discount::numeric)), 2) AS arredondar_total,
       sum(round(unit_price::numeric * quantity *
                (1 - discount::numeric), 2)) AS arredondar_itens
FROM public.order_details;
```

arredondar_total	arredondar_itens
1.265.793,04	1.265.793,29

Arredondar cada item antes de somar acrescenta **0,25** ao total. Por isso, todo o projeto calcula com decimais exatos e arredonda apenas na apresentação. Essa regra se mostrou decisiva na comparação entre os bancos.

4.9 Preço e endereço pertencem ao pedido

Consulta 8 — Preço do item comparado ao preço do catálogo.

```
SELECT count(*) AS itens,
       count(*) FILTER
         (WHERE d.unit_price <> p.unit_price) AS precos_diferentes
FROM public.order_details d
JOIN public.products p ON p.product_id = d.product_id;
```

itens	precos_diferentes
2155	662

Em **662 dos 2.155 itens (30,7%)**, o preço da venda é diferente do preço atual do catálogo. Com o endereço acontece o mesmo: em 82 dos 830 pedidos, o endereço de envio difere do cadastro do cliente. Conclusão: preço e endereço são fatos da venda e precisam ser guardados no pedido, não buscados no cadastro.

4.10 Avaliação

A base é adequada para análises descritivas: os relacionamentos fecham e os valores estão dentro das faixas. Os cuidados que guiaram o restante do projeto são quatro: respeitar o significado dos nulos, manter preço e endereço do pedido, calcular com decimais exatos e não tratar anos incompletos como completos.

5. Arquitetura híbrida

A exploração mostrou dois tipos de necessidade. Para registrar vendas, a base precisa de regras rígidas: preço não negativo, desconto abaixo de 100%, item ligado a um pedido existente. Para ler um pedido completo, o ideal é ter tudo num só lugar. O projeto testa as duas formas lado a lado, com um fluxo de mão única.

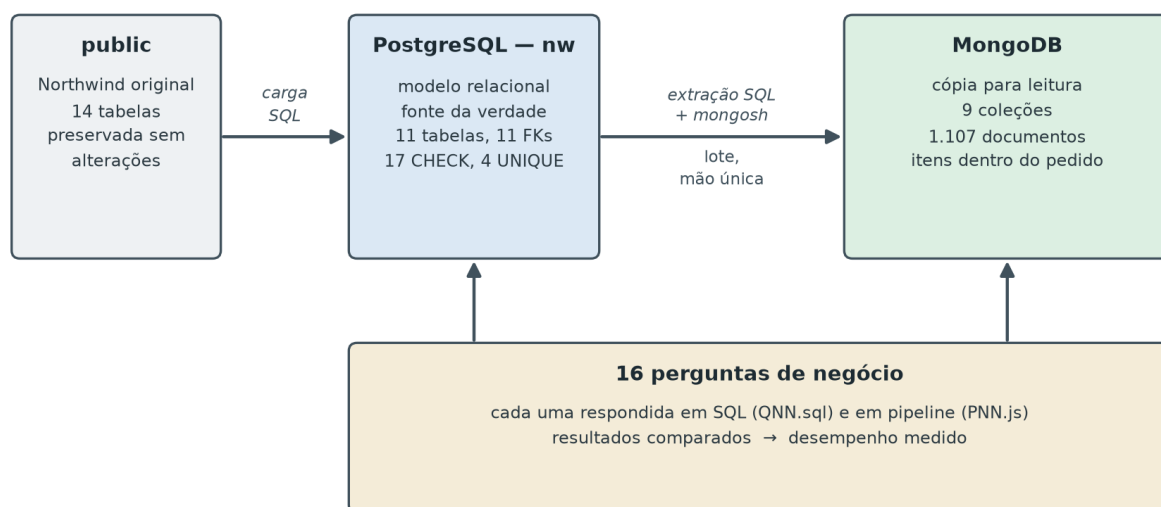


Figura 2 — Fluxo de dados do projeto.

O schema **public** guarda a base original sem alterações. O schema **nw** é o modelo relacional do projeto e funciona como fonte da verdade. A partir dele, uma extração em SQL gera os documentos, que o mongosh carrega no MongoDB. O MongoDB não recebe escritas diretas: é uma cópia orientada à leitura, reconstruída a partir do PostgreSQL quando necessário.

A mão única evita que o mesmo dado tenha dois donos. Se os dois bancos aceitassem alterações, seria preciso decidir qual vale em caso de conflito. A carga em lote, e não em tempo real, é suficiente porque nenhuma das perguntas do projeto depende de dados atualizados ao segundo.

6. Modelo relacional

O schema **nw** tem 11 tabelas e 3.311 linhas. As duas tabelas demográficas vazias e `us_states` ficaram apenas na fonte. Em relação à base original, o modelo corrige tipos e acrescenta regras:

Item	Implementação
Chaves	11 chaves primárias e 11 chaves estrangeiras
Regras de domínio	17 restrições CHECK (valores não negativos, desconto de 0 a menos de 1, quantidade positiva, envio não anterior ao pedido, entre outras)
Unicidade	4 restrições UNIQUE, como nome de categoria e produto por fornecedor
Tipos	Dinheiro em <code>numeric(10,2)</code> , desconto em <code>numeric(4,3)</code> , discontinued como boolean
Índices	<code>orders(customer_id)</code> , <code>orders(employee_id)</code> , <code>orders(order_date)</code> e <code>order_items(product_id)</code>

6.1 Normalização

Os itens ficam numa tabela própria, `order_items`, com chave (`order_id`, `product_id`). Isso evita colunas repetidas como `produto_1` e `produto_2` e permite qualquer número de itens por pedido (primeira forma normal).

O preço do item depende da venda, não só do produto: o mesmo produto foi vendido a preços diferentes em pedidos diferentes. Por isso `unit_price` fica em `order_items`, e não é buscado em `products`. O mesmo raciocínio mantém o endereço de envio em `orders`. Já o nome do cliente depende apenas do cliente e fica só em `customers`, sem cópias nos pedidos.

O campo `quantity_per_unit` (por exemplo, “10 boxes x 30 bags”) foi mantido como texto descritivo, porque o projeto não precisa separar quantidade e embalagem.

6.2 Modelo lógico

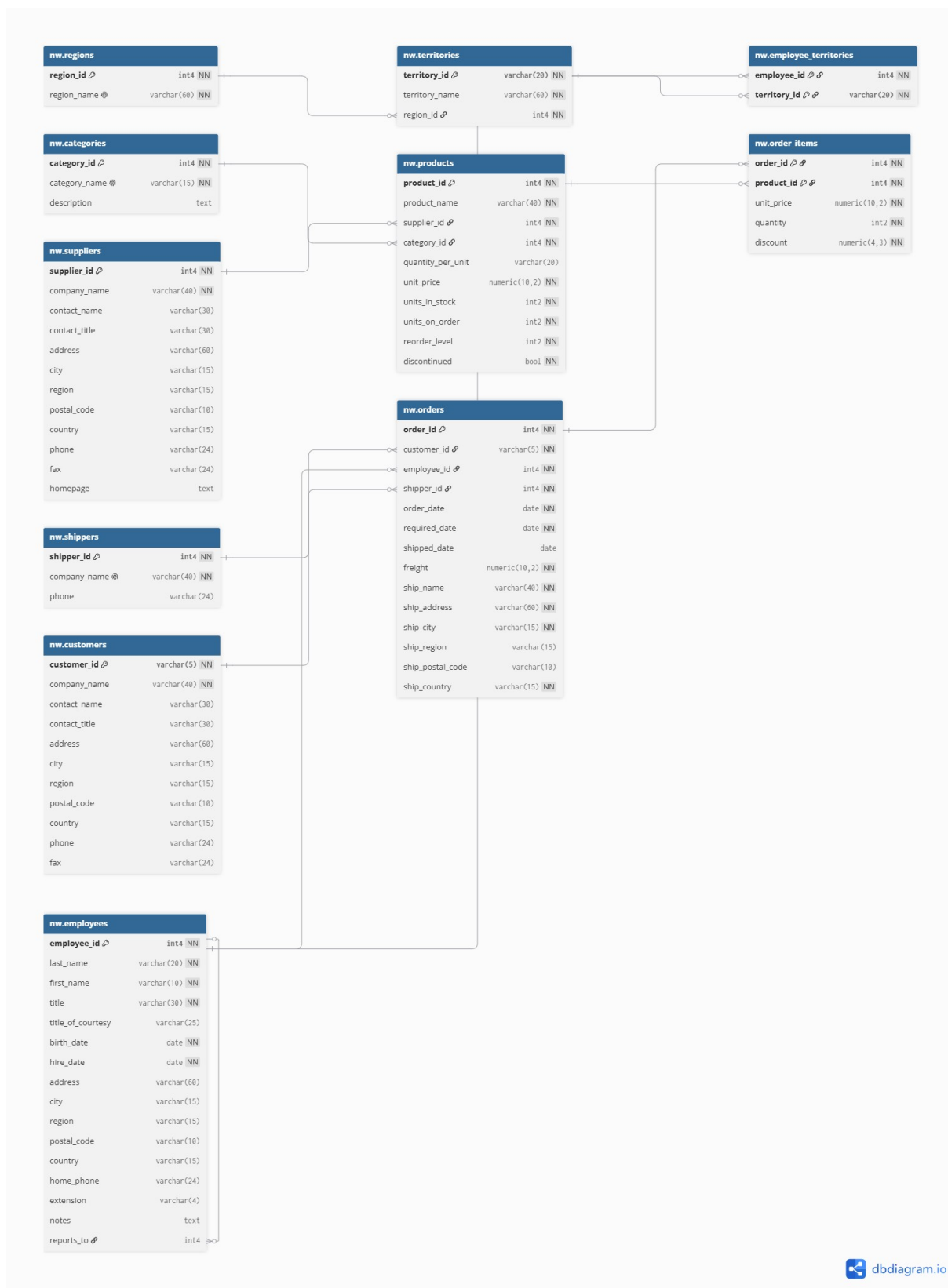


Figura 3 — Modelo lógico do schema nw, com tipos e chaves.

As chaves estrangeiras garantem que um item aponte para um pedido existente, mas não garantem que todo pedido tenha ao menos um item. Essa regra foi verificada por consulta (capítulo 4). Os índices foram escolhidos pelos acessos mais frequentes das análises: pedidos por cliente, por funcionário e por data, e itens por produto.

7. Modelo documental

No MongoDB, a pergunta central de modelagem é o que deve ficar junto. A regra adotada foi: **incorporar o que pertence ao pedido e é lido com ele; referenciar o que existe por conta própria** [5].

Coleção	Documentos	Conteúdo
orders	830	Pedido com datas, frete, endereço de envio (objeto shipping) e array items com os itens
products	77	Produto com estoque, categoria e fornecedor (id e nome)
customers	91	Cliente com endereço agrupado em objeto
employees	9	Funcionário, referência ao superior e array de territórios
categories, suppliers, shippers	8, 29, 6	Cadastros completos, inclusive sem movimento
territories, regions	53, 4	Malha comercial completa

As 3.311 linhas viraram **1.107 documentos** porque duas tabelas deixaram de existir como coleção: os 2.155 itens foram para dentro dos pedidos e os 49 vínculos de território, para dentro dos funcionários (3.311 – 2.155 – 49 = 1.107).

7.1 O que foi incorporado e o que foi referenciado

Itens dentro do pedido. Um item não existe sem o pedido e quase sempre é lido com ele. Incorporar elimina a junção mais frequente das análises.

Clientes, produtos e funcionários em coleções próprias. Eles existem independentemente de uma venda e aparecem em muitos pedidos. Os pedidos guardam apenas o identificador, como `customer_id` e `employee_id`. Colocar os pedidos dentro do cliente criaria um array que cresce sem limite.

Cópias de leitura. Cada item guarda o id, o nome e a categoria do produto. Isso deixa o pedido legível sem consultar `products`, ao custo de atualizar as cópias quando o cadastro mudar. Essas cópias refletem o cadastro no momento da carga; o preço e o endereço, ao contrário, vêm da própria venda.

7.2 O pedido 10248 nos dois modelos

No PostgreSQL, o pedido 10248 ocupa uma linha em `orders` e três em `order_items`. No MongoDB, é um único documento (abaixo, com o primeiro dos três itens):

```
{
  _id: 10248,
  customer_id: "VINET",
  order_date: ISODate("1996-07-04T00:00:00Z"),
  freight: Decimal128("32.38"),
  shipping: {
    name: "Vins et alcools Chevalier",
    street: "59 rue de l'Abbaye",
    city: "Reims", region: null,
    postal_code: "51100", country: "France"
  },
  items: [{
    product: {
      _id: 11, name: "Queso Cabrales",
      category: { _id: 4, name: "Dairy Products" }
    },
    unit_price: Decimal128("14.00"),
    quantity: 12, discount: Decimal128("0.000")
  }]
}
```

Os três itens valem 168,00 + 98,00 + 174,00 = **440,00**, sem o frete de 32,38.

7.3 Tipos

Identificadores e quantidades são inteiros; códigos de cliente e território continuam texto. Preços, descontos e frete usam **Decimal128**, o tipo decimal exato do MongoDB [6], para que nenhum valor passe por ponto flutuante. As datas viraram BSON Date à meia-noite UTC, já que o PostgreSQL guarda só o dia. Campos sem valor ficaram como null, e não ausentes, para que as consultas tratem os dois bancos da mesma forma.

8. Transformação, carga e validação dos documentos

8.1 Do relacional ao documento

Os documentos foram montados no próprio PostgreSQL, com funções JSON, numa transação REPEATABLE READ para que todas as tabelas fossem lidas no mesmo instante. A operação central agrupa os itens de cada pedido num array.

Consulta 9 — Agrupamento dos itens do pedido 10248.

```
SELECT o.order_id,
       jsonb_agg(jsonb_build_object(
         'product_id', i.product_id,
         'unit_price', i.unit_price,
         'quantity', i.quantity,
         'discount', i.discount
       ) ORDER BY i.product_id) AS items
FROM   nw.orders o
JOIN   nw.order_items i USING (order_id)
WHERE  o.order_id = 10248
GROUP BY o.order_id;
```

```
{
  "order_id": 10248,
  "items": [
    {"product_id": 11, "unit_price": 14.00,
     "quantity": 12, "discount": 0.000},
    {"product_id": 42, "unit_price": 9.80,
     "quantity": 10, "discount": 0.000},
    {"product_id": 72, "unit_price": 34.80,
     "quantity": 5, "discount": 0.000}
  ]
}
```

Na transformação completa, cada pedido recebe também datas, frete, endereço e os dados de leitura do produto. Os territórios dos funcionários seguem a mesma lógica. Os valores decimais viajam em Extended JSON, como {"\$numberDecimal": "14.00"}, e chegam ao MongoDB já como Decimal128.

8.2 Carga repetível

O mongosh grava os documentos com substituição por identificador:

```
db.orders.replaceOne(
  { _id: documento._id },
  documento,
  { upsert: true }
);
```

Se o documento não existe, ele é inserido; se existe, é substituído por inteiro. Rodar a carga duas vezes não duplica nada:

Execução	Inseridos	Modificados
Primeira carga	1.107	0
Repetição	0	0

Se a origem perder um registro que ainda existe no MongoDB, a carga para antes de gravar, para revisão. O MongoDB roda em modo standalone, sem transação que envolva todas as coleções; se a carga for interrompida, a recuperação é repeti-la inteira e conferir de novo.

8.3 Validadores

Sem regras, o MongoDB aceitaria um preço em texto ou uma quantidade zero. As nove coleções receberam validadores JSON Schema, inclusive para objetos e arrays aninhados.

Campo	Regra
Preços e frete	Decimal128, maior ou igual a zero
Quantidade	Inteiro, maior ou igual a um

Campo	Regra
Desconto	Decimal128, de zero a menos de um
Datas	BSON Date; data de envio admite null
Endereço de envio	Objeto obrigatório com os seis campos
Itens	Array de objetos com produto, preço, quantidade e desconto

```
discount: {
  bsonType: "decimal",
  minimum: Decimal128("0"),
  maximum: Decimal128("1"),
  exclusiveMaximum: true
}
```

Cinco alterações inválidas foram testadas numa base isolada: preço como texto, quantidade zero, desconto de 100%, data como texto e pedido sem endereço. **As cinco foram rejeitadas** (erro 121) e o documento original ficou intacto.

O validador confere a forma do documento, mas não sabe se o `customer_id` existe em `customers`. Essa checagem, que no PostgreSQL é feita pela chave estrangeira, foi feita por consulta depois da carga.

9. Índices no MongoDB

Além do índice automático em `_id`, foram criados três índices, cada um ligado a um acesso concreto:

Índice	Campos	Acesso atendido
cliente_data	customer_id, order_date	Pedidos de um cliente num período, em ordem de data
produto_no_pedido	items.product._id	Pedidos que contêm um produto
categoria_produto	category._id	Produtos de uma categoria

No índice composto, o cliente vem primeiro porque é filtrado por igualdade; a data vem depois e atende ao intervalo e à ordenação. O índice em `items.product._id` é multikey: ele indexa cada produto do array de itens.

```
db.orders.find({
  customer_id: "VINET",
  order_date: {
    $gte: ISODate("1996-01-01T00:00:00Z"),
    $lt: ISODate("1997-01-01T00:00:00Z")
  }
}).sort({ order_date: 1 }).explain("executionStats");
```

Medida	Resultado
Plano	FETCH → IXSCAN
Índice usado	cliente_data
Documentos examinados / retornados	3 / 3

Sem índice, o MongoDB leria os 830 pedidos para devolver três. Com ele, lê exatamente os três. O plano de P03 (capítulo 13) mostrou também o uso de `produto_no_pedido`. Índices custam espaço e tornam as escritas mais lentas, por isso não foram criados para todos os campos. Como o capítulo 16 mostra, faltaram dois que fariam diferença.

10. Conferência da migração

Contar documentos não basta: duas tabelas com dez linhas podem ter preços diferentes. A prova adotada foi o caminho inverso. Os documentos gravados no MongoDB foram lidos de volta, desmontados em linhas (itens separados dos pedidos, endereços desagrupados, territórios reconstituídos) e comparados com as 11 tabelas de nw, campo a campo, nos dois sentidos, com EXCEPT ALL. Esse procedimento foi escrito separadamente da carga, para não repetir um eventual erro dela.

Tabela reconstruída	Origem	MongoDB	Diferenças
regions	4	4	0
territories	53	53	0
categories	8	8	0
shippers	6	6	0
suppliers	29	29	0
customers	91	91	0
employees	9	9	0
employee_territories	49	49	0
products	77	77	0
orders	830	830	0
order_items	2.155	2.155	0
Total	3.311	3.311	0

As 3.311 linhas voltaram sem nenhuma diferença. A soma dos itens também foi calculada nos dois bancos, com decimais exatos, e deu o mesmo valor: **1265793.03950**, apresentado como 1.265.793,04.

Para ter certeza de que a comparação não escondia diferenças pequenas, uma cópia dos dados recebeu o preço 14.004 no lugar de 14.00. A comparação acusou a mudança, ou seja, detecta diferenças menores que um centavo. A conferência foi repetida nas Entregas 3 e 4, sempre com zero diferenças.

11. Regras comuns às análises

As 16 perguntas foram respondidas duas vezes: uma consulta SQL (QNN.sql) e um pipeline MongoDB (PNN.js) para cada uma. Para que os dois lados pudessem ser comparados número a número, as regras abaixo foram fixadas antes de escrever qualquer código.

Regra	Definição
Período	De 01/07/1996 (incluído) a 01/06/1998 (excluído), que cobre todos os pedidos
Valor	preço praticado $\times$ quantidade $\times$ (1 – desconto), sem frete, com decimais exatos
Arredondamento	Só na apresentação, com duas casas; nenhuma soma usa valores arredondados
Ticket	Valor dos pedidos $\div$ número de pedidos; os itens são somados por pedido antes
Referência de recência	06/05/1998, último pedido; inativo é quem não compra desde antes de 06/11/1997
Meses vazios	As séries mensais têm os 23 meses, com zero onde não houver pedidos
Cadastros sem movimento	Clientes, funcionários e transportadoras sem pedidos aparecem com zero
Desempate	Sempre pelo identificador, para que a ordem seja a mesma nos dois bancos

Duas regras merecem destaque. Primeiro, a **curva ABC** classifica pelo valor acumulado antes de cada cliente: é classe A quem entra enquanto o acumulado está abaixo de 80% do total, B abaixo de 95% e C o restante. Segundo, o **RFM** usa faixas fixas e didáticas: recência até 30 dias = 3, até 90 = 2, acima = 1; frequência de pelo menos 10 pedidos = 3, pelo menos 5 = 2; valor de pelo menos 10.000 = 3, pelo menos 5.000 = 2. Quem nunca comprou recebe zero.

11.1 Como os resultados foram conferidos

Cada par foi comparado linha a linha e campo a campo, na ordem em que os bancos devolvem os dados. Somas de dinheiro precisam ser exatamente iguais; médias e percentuais admitem diferença de até 0,000000000001, por causa das dízimas.

SQL e MongoDB poderiam errar a mesma regra e ainda concordar entre si. Por isso, houve mais duas verificações:

- **Um terceiro cálculo**, em Python puro, a partir das linhas brutas de nw, sem SQL analítico. Os 16 resultados bateram.
- **Testes com quatro pedidos inventados**, numa base isolada, cujo resultado foi calculado à mão: mês sem vendas, compra exatamente no dia do corte de inatividade, item de 0,005, pedido sem data de envio e empate no ranking. Ao todo, 38 verificações, todas aprovadas.

12. Views e procedures

12.1 Views para relatórios

O cálculo do valor aparece em quase todas as análises. Para não repeti-lo em cada consulta, ele foi centralizado em quatro views no schema **nw_analytics**:

View	Uma linha por	Usada em
vw_item_valor	item, com valor bruto, desconto e valor final	Q01, Q07, Q14, Q16 e procedure de vendas
vw_pedido_valor	pedido, com itens, unidades e valor	Q02, Q04, Q08 a Q11, Q15 e procedure de equipe
vw_vendas_mensais	mês com pedidos, com valor e ticket	Relatório direto
vw_envio_transportadora	transportadora, com envios, datas ausentes e média de dias	Relatório direto

Uma view guarda a consulta, não os dados: cada leitura recalcula a partir das tabelas. As views antigas do schema nw, que arredondavam cada item, foram mantidas sem uso.

12.2 Procedures

Três procedures em PL/pgSQL recebem parâmetros e devolvem o resultado por um cursor. Todas validam a entrada: período invertido, número de meses não positivo e gestor inexistente são recusados com erro.

Procedure	Parâmetros	Resultado
sp_resumo_vendas_periodo	início, fim	Pedidos, valor e ticket por categoria no período
sp_clientes_inativos	data de referência, meses	Clientes inativos e clientes sem compras
sp_desempenho_equipe	gestor, início, fim	Vendas próprias e da equipe do gestor

A chamada abre uma transação, executa a procedure, lê o cursor e encerra:

```
BEGIN;
CALL nw_analytics.sp_clientes_inativos('1998-05-06', 6, 'inativos');
FETCH ALL FROM inativos;
COMMIT;
```

Os resultados das três foram comparados com consultas independentes. Os códigos completos estão no Apêndice B.

Saída de CALL sp_resumo_vendas_periodo('1996-07-01', '1998-06-01', ...):

category_id	categoria	pedidos	valor	ticket_categoria
1	Beverages	354	267.868,18	756,69
2	Condiments	193	106.047,09	549,47
3	Confections	295	167.357,23	567,31
4	Dairy Products	303	234.507,29	773,95
5	Grains/Cereals	182	95.744,59	526,07
6	Meat/Poultry	161	163.022,36	1.012,56
7	Produce	129	99.984,58	775,07
8	Seafood	291	131.261,74	451,07

A chamada CALL sp_clientes_inativos('1998-05-06', 6, ...) devolveu 7 clientes, os mesmos de Q06, na mesma ordem (tabela completa na seção 13.6).

Saída de CALL sp_desempenho_equipe(2, '1996-07-01', '1998-06-01', ...):

gestor_id	membros	valor proprio	pedidos equipe	valor equipe
2	9	166.537,76	830	1.265.793,04

13. Consultas analíticas e resultados

Cada seção traz a pergunta, a regra de cálculo, os recursos usados em cada banco, o resultado e o que ele significa. As tabelas mostram a saída validada, idêntica nos dois bancos, com decimais arredondados só na apresentação. Quando o resultado é longo, o recorte exibido está indicado. Os 32 códigos completos estão no Apêndice A.

13.1 Q01/P01 — Valor e participação por categoria

Pergunta: Quais categorias concentram o valor dos pedidos?

Cálculo: Relacionar itens ao pedido e à categoria, somar quantidade e valor e dividir cada valor pelo total geral.

Recursos: SQL: JOIN, GROUP BY e SUM OVER. MongoDB: \$unwind, \$group e \$setWindowFields.

Resultado: 8 linhas; todas exibidas.

categoria (ID)	categoria	unidades	valor	participação (%)
1	Beverages	9532	267.868,18	21,16
2	Condiments	5298	106.047,09	8,38
3	Confections	7906	167.357,23	13,22
4	Dairy Products	9149	234.507,29	18,53
5	Grains/Cereals	4562	95.744,59	7,56
6	Meat/Poultry	4199	163.022,36	12,88
7	Produce	2990	99.984,58	7,90
8	Seafood	7681	131.261,74	10,37

Leitura: Beverages lidera com 21,16% do valor (267.868,18), seguida por Dairy Products com 18,53%. Juntas, as duas categorias somam quase 40% de tudo o que foi vendido.

Cuidado: A participação mede peso no valor dos pedidos, não rentabilidade. Os grupos são definidos pelo ID da categoria e o percentual é calculado depois da agregação, sobre o mesmo recorte.

13.2 Q02/P02 — Pedidos e ticket médio mensal

Pergunta: Como variam o número de pedidos e o ticket médio mensal?

Cálculo: Somar itens por pedido, agrupar pedidos por mês e completar o calendário. Ticket = valor / pedidos.

Recursos: SQL: CTEs, generate_series e agregação em dois níveis. MongoDB: \$map, \$group, \$facet e calendário por \$dateAdd.

Resultado: 23 linhas; todas exibidas.

mês	pedidos	valor	ticket
1996-07-01	22	27.861,90	1.266,45
1996-08-01	25	25.485,28	1.019,41
1996-09-01	23	26.381,40	1.147,02
1996-10-01	26	37.515,73	1.442,91
1996-11-01	25	45.600,05	1.824,00
1996-12-01	31	45.239,63	1.459,34
1997-01-01	33	61.258,07	1.856,31
1997-02-01	29	38.483,64	1.327,02
1997-03-01	30	38.547,22	1.284,91
1997-04-01	31	53.032,95	1.710,74
1997-05-01	32	53.781,29	1.680,67
1997-06-01	30	36.362,80	1.212,09
1997-07-01	33	51.020,86	1.546,09

mês	pedidos	valor	ticket
1997-08-01	33	47.287,67	1.432,96
1997-09-01	37	55.629,24	1.503,49
1997-10-01	38	66.749,23	1.756,56
1997-11-01	34	43.533,81	1.280,41
1997-12-01	48	71.398,43	1.487,47
1998-01-01	55	94.222,11	1.713,13
1998-02-01	54	99.415,29	1.841,02
1998-03-01	73	104.854,16	1.436,36
1998-04-01	74	123.798,68	1.672,96
1998-05-01	14	18.333,63	1.309,55

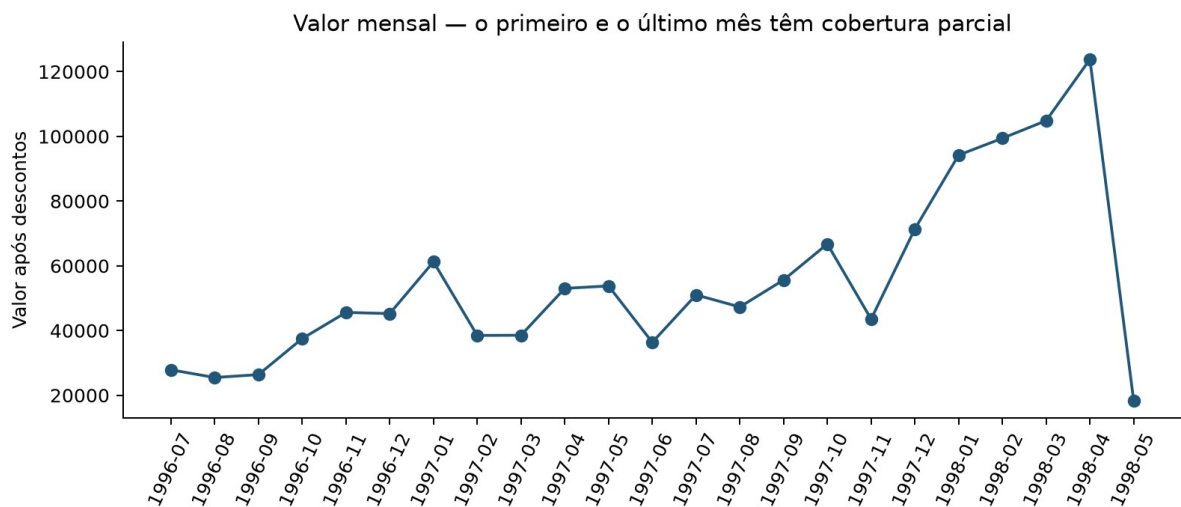


Figura 4 — Valor mensal após descontos (Q02/P02). Julho de 1996 começa no dia 4 e maio de 1998 termina no dia 6.

Leitura: O volume mensal mais que dobrou entre o início e o fim da série: de 33 pedidos em janeiro de 1997 para 74 em abril de 1998, o maior valor mensal (123.798,68). Maio de 1998 aparece baixo porque a base termina no dia 6.

Cuidado: O ticket é calculado por pedido: a média dos valores dos itens mediria o item médio, não o pedido médio. Um mês sem pedidos teria valor zero e ticket indefinido (nulo).

13.3 Q03/P03 — Produtos sem vendas no período

Pergunta: Quais produtos não foram vendidos no período?

Cálculo: Começar pelo cadastro completo e verificar se existe pelo menos um item em pedido dentro da janela.

Recursos: SQL: NOT EXISTS e junções cadastrais. MongoDB: \$lookup com \$limit e filtro de array vazio.

Resultado: nenhuma linha nos dois bancos.

Leitura: Nenhum produto ficou sem venda: os 77 aparecem em pelo menos um pedido do período. Um resultado vazio também é uma resposta.

Cuidado: O resultado vazio vale apenas para o recorte completo; em janelas menores podem surgir produtos sem venda. Os testes sintéticos confirmam que a consulta lista esses produtos quando existem.

13.4 Q04/P04 — Indicadores por funcionário

Pergunta: Como se distribuem os pedidos entre os funcionários?

Cálculo: Preservar o cadastro de funcionários, somar pedidos e valor de cada um e calcular ticket e participação.

Recursos: SQL: LEFT JOIN, agregação e janela global. MongoDB: \$lookup, \$group e \$setWindowFields.

Resultado: 9 linhas; todas exibidas.

funcionário (ID)	funcionário	pedidos	valor	ticket	participação (%)
1	Nancy Davolio	123	192.107,60	1.561,85	15,18
2	Andrew Fuller	96	166.537,76	1.734,77	13,16
3	Janet Leverling	127	202.812,84	1.596,95	16,02
4	Margaret Peacock	156	232.890,85	1.492,89	18,40
5	Steven Buchanan	42	68.792,28	1.637,91	5,43
6	Michael Suyama	67	73.913,13	1.103,18	5,84
7	Robert King	72	124.568,24	1.730,11	9,84
8	Laura Callahan	104	126.862,28	1.219,83	10,02
9	Anne Dodsworth	43	77.308,07	1.797,86	6,11

Leitura: Margaret Peacock tem o maior valor registrado: 232.890,85 em 156 pedidos, 18,40% do total. Sem dados de carteira ou território, a comparação descreve vendas, e não produtividade.

Cuidado: Os valores descrevem vendas registradas por funcionário. Sem informação de carteira, território ou tempo de atuação, a diferença entre eles não mede produtividade.

13.5 Q05/P05 — Estoque crítico e cobertura por categoria

Pergunta: Quais categorias têm produtos abaixo do ponto de reposição?

Cálculo: Comparar estoque com reorder_level, somar a diferença e verificar se estoque mais encomendado cobre o ponto.

Recursos: SQL: Comparação entre colunas, FILTER e array_agg. MongoDB: \$expr, \$set, \$group e \$push.

Resultado: 7 linhas; todas exibidas.

categoria (ID)	categoria	produtos	déficit	cobertura insuficiente	produtos (IDs)
1	Beverages	3	31	1	2, 43, 70
2	Condiments	2	28	0	3, 66
3	Confections	4	26	0	21, 48, 49, 68
4	Dairy Products	3	44	0	11, 31, 32
5	Grains/Cereals	2	17	0	56, 64
7	Produce	1	1	0	74
8	Seafood	3	29	1	30, 37, 45

Leitura: Há 18 produtos abaixo do ponto, em sete categorias. Em dois deles, nem o estoque somado ao encomendado alcança o ponto. Um dos 18, Chang (ID 2), está descontinuado, o que mostra por que o sinal precisa ser conferido antes de uma compra.

Cuidado: O estoque é o estado cadastrado, sem reconstrução histórica. O indicador precisa ser conferido com o status do produto antes de virar recomendação de compra.

13.6 Q06/P06 — Clientes inativos e sem compras

Pergunta: Quais clientes estão sem compra há mais de seis meses?

Cálculo: Para cada cliente, obter a última compra até 06/05/1998. Selecionar data anterior a 06/11/1997 ou ausência de compras.

Recursos: SQL: LEFT JOIN, MAX e comparação de datas. MongoDB: \$lookup, \$dateDiff e tratamento de null.

Resultado: 7 linhas; todas exibidas.

cliente (código)	cliente	última compra	recência (dias)	situação
CENTC	Centro comercial Moctezuma	1996-07-18	657	inativo
FAMIA	Familia Arquibaldo	1997-10-31	187	inativo
FISSA	FISSA Fabrica Inter. Salchichas S.A.	—	—	sem compras
HUNGC	Hungry Coyote Import Store	1997-09-08	240	inativo
LAZYK	Lazy K Kountry Store	1997-05-22	349	inativo
MEREP	Mère Paillard	1997-10-30	188	inativo
PARIS	Paris spécialités	—	—	sem compras

Leitura: Cinco clientes não compram desde antes do corte e dois nunca compraram (FISSA e PARIS). Entre os inativos está MEREP, nono maior cliente em valor e classe A na curva ABC, sem compras desde 30/10/1997.

Cuidado: A referência 06/05/1998 é o último pedido da base; a data atual tornaria todos os clientes inativos. Seis meses de calendário não equivalem necessariamente a 180 dias.

13.7 Q07/P07 — Top cinco produtos por categoria

Pergunta: Quais são os cinco produtos de maior valor em cada categoria?

Cálculo: Agregar por produto e categoria, ordenar por valor decrescente e ID crescente, numerar dentro de cada categoria e manter posições de 1 a 5.

Recursos: SQL: ROW_NUMBER com PARTITION BY. MongoDB: \$setWindowFields e soma cumulativa de 1.

Resultado: 40 linhas; exibido o primeiro colocado de cada categoria.

categoria (ID)	produto (ID)	valor	posição
1	38	141.396,74	1
2	63	16.701,10	1
3	62	47.234,97	1
4	59	71.155,70	1
5	56	42.593,06	1
6	29	80.368,67	1
7	51	41.819,65	1
8	18	29.171,88	1

Leitura: O produto 38 (Côte de Blaye) lidera Beverages com 141.396,74, cerca de seis vezes o segundo colocado da categoria e 52,79% do valor de Beverages. Sozinho, ele responde por 11,17% do valor total do período, o que torna a categoria dependente de um único item.

Cuidado: O ranking é por valor, não por unidades vendidas. Com ROW_NUMBER, empates na quinta posição são resolvidos pelo menor ID; RANK poderia devolver mais de cinco linhas.

13.8 Q08/P08 — Valor mensal acumulado

Pergunta: Qual é o valor acumulado ao longo dos meses?

Cálculo: Construir a série mensal e somar os valores da primeira linha até a linha corrente.

Recursos: SQL: SUM OVER com ROWS UNBOUNDED PRECEDING. MongoDB: \$setWindowFields com janela até current.

Resultado: 23 linhas; todas exibidas.

mês	valor	acumulado
1996-07-01	27.861,90	27.861,90
1996-08-01	25.485,28	53.347,17
1996-09-01	26.381,40	79.728,57
1996-10-01	37.515,73	117.244,30
1996-11-01	45.600,05	162.844,34
1996-12-01	45.239,63	208.083,97
1997-01-01	61.258,07	269.342,04
1997-02-01	38.483,64	307.825,68
1997-03-01	38.547,22	346.372,90
1997-04-01	53.032,95	399.405,85
1997-05-01	53.781,29	453.187,14
1997-06-01	36.362,80	489.549,94
1997-07-01	51.020,86	540.570,80
1997-08-01	47.287,67	587.858,47
1997-09-01	55.629,24	643.487,71
1997-10-01	66.749,23	710.236,94
1997-11-01	43.533,81	753.770,75
1997-12-01	71.398,43	825.169,17
1998-01-01	94.222,11	919.391,28
1998-02-01	99.415,29	1.018.806,57
1998-03-01	104.854,16	1.123.660,73
1998-04-01	123.798,68	1.247.459,41
1998-05-01	18.333,63	1.265.793,04

Leitura: O acumulado termina em 1.265.793,04, o mesmo total calculado diretamente sobre os itens, o que confirma que nenhum mês ficou de fora ou foi contado duas vezes.

Cuidado: Um acumulado crescente não indica crescimento mensal: ele sobe sempre que o mês tem valor positivo. O total é a última linha, não a soma dos acumulados.

13.9 Q09/P09 — Variação mensal do valor

Pergunta: Quanto o valor mudou em relação ao mês anterior?

Cálculo: Obter o valor do mês anterior, subtrair do atual e dividir a diferença pelo anterior.

Recursos: SQL: LAG e NULLIF. MongoDB: \$shift e \$cond.

Resultado: 23 linhas; todas exibidas.

mês	valor	mês anterior	variação	variação (%)
1996-07-01	27.861,90	—	—	—
1996-08-01	25.485,28	27.861,90	-2.376,62	-8,53
1996-09-01	26.381,40	25.485,28	896,13	3,52
1996-10-01	37.515,73	26.381,40	11.134,33	42,21
1996-11-01	45.600,05	37.515,73	8.084,32	21,55
1996-12-01	45.239,63	45.600,05	-360,42	-0,79
1997-01-01	61.258,07	45.239,63	16.018,44	35,41
1997-02-01	38.483,64	61.258,07	-22.774,44	-37,18
1997-03-01	38.547,22	38.483,64	63,59	0,17
1997-04-01	53.032,95	38.547,22	14.485,73	37,58

mês	valor	mês anterior	variação	variação (%)
1997-05-01	53.781,29	53.032,95	748,34	1,41
1997-06-01	36.362,80	53.781,29	-17.418,49	-32,39
1997-07-01	51.020,86	36.362,80	14.658,06	40,31
1997-08-01	47.287,67	51.020,86	-3.733,19	-7,32
1997-09-01	55.629,24	47.287,67	8.341,57	17,64
1997-10-01	66.749,23	55.629,24	11.119,98	19,99
1997-11-01	43.533,81	66.749,23	-23.215,42	-34,78
1997-12-01	71.398,43	43.533,81	27.864,62	64,01
1998-01-01	94.222,11	71.398,43	22.823,68	31,97
1998-02-01	99.415,29	94.222,11	5.193,18	5,51
1998-03-01	104.854,16	99.415,29	5.438,87	5,47
1998-04-01	123.798,68	104.854,16	18.944,53	18,07
1998-05-01	18.333,63	123.798,68	-105.465,05	-85,19

Leitura: Agosto de 1996 ficou 8,53% abaixo de julho. A maior alta ocorreu em dezembro de 1997 (+64,01%). A queda de 85,19% em maio de 1998 não indica retração: a base só registra pedidos até 06/05/1998.

Cuidado: Quando o mês anterior tem valor zero, o percentual fica indefinido. O calendário completo impede que um mês sem vendas seja pulado e que meses não consecutivos sejam comparados.

13.10 Q10/P10 — Curva ABC de clientes

Pergunta: Como os clientes se distribuem pela curva ABC de valor?

Cálculo: Somar valor por cliente, ordenar, calcular acumulado e aplicar os limites de 80% e 95% ao acumulado anterior.

Recursos: SQL: CTEs, janela cumulativa e CASE. MongoDB: \$lookup, duas janelas e \$switch.

Resultado: 91 linhas; exibidas as cinco primeiras na ordenação definida.

cliente (código)	cliente	pedidos	valor	acumulado	classe
QUICK	QUICK-Stop	28	110.277,31	110.277,31	A
ERNSH	Ernst Handel	30	104.874,98	215.152,28	A
SAVEA	Save-a-lot Markets	31	104.361,95	319.514,23	A
RATTC	Rattlesnake Canyon Grocery	18	51.097,80	370.612,03	A
HUNGO	Hungry Owl All-Night Grocers	19	49.979,91	420.591,94	A

Resumo das 91 linhas por classe:

classe	clientes	valor	participação (%)
A	34	1.013.547,90	80,07
B	27	192.826,66	15,23
C	28	59.418,47	4,69
sem compras	2	0,00	0,00

Leitura: 34 clientes formam a classe A e concentram 80,07% do valor; 27 ficam na B e 28 na C. QUICK-Stop lidera com 110.277,31. Os dois clientes sem compras ficam fora das classes.

Cuidado: A classificação depende da concentração do valor, não da quantidade de clientes; por isso NTILE não seria equivalente. O cliente que cruza um limite permanece na classe que estava sendo completada.

13.11 Q11/P11 — RFM descritivo dos clientes

Pergunta: Qual é o perfil de recência, frequência e valor de cada cliente?

Cálculo: Recência são dias desde a última compra; frequência é quantidade de pedidos; valor é a soma após descontos. Aplicar faixas fixas documentadas.

Recursos: SQL: CTE e CASE. MongoDB: \$lookup, \$dateDiff e condições.

Resultado: 91 linhas; exibidas as cinco primeiras na ordenação definida.

cliente (código)	recência (dias)	frequência	valor	R
ALFKI	27	6	4.273,00	3
ANATR	63	4	1.402,95	2
ANTON	98	7	7.023,98	1
AROUT	26	13	13.390,65	3
BERGS	63	18	24.927,58	2

cliente (código)	F	M
ALFKI	2	1
ANATR	1	1
ANTON	2	2
AROUT	3	3
BERGS	3	3

Leitura: 28 clientes têm a combinação máxima R=3, F=3 e M=3: compraram no último mês, fizeram pelo menos dez pedidos e gastaram pelo menos 10.000. Outros 12 compraram recentemente, mas com frequência média e valor baixo (R=3, F=2, M=1), um grupo com espaço para crescer.

Cuidado: As faixas são convenções didáticas, não um modelo estatístico. Clientes sem compras recebem escores zero e recência nula, sem data inventada.

13.12 Q12/P12 — Produtos comprados juntos

Pergunta: Quais produtos aparecem juntos no mesmo pedido?

Cálculo: Formar combinações de dois IDs diferentes no pedido, mantendo apenas A menor que B, contar pedidos por par e dividir pelo total de pedidos.

Recursos: SQL: Autojunção de itens e agregação. MongoDB: Expansão de arrays, \$match e \$group.

Resultado: 1535 linhas; exibidas as cinco primeiras na ordenação definida.

produto A	produto B	pedidos	pedidos no período	suporte (%)
21	61	8	830	0,96
16	31	7	830	0,84
16	60	6	830	0,72
16	62	6	830	0,72
30	54	6	830	0,72

Leitura: Foram encontrados 1.535 pares diferentes. O mais frequente, produtos 21 e 61, aparece junto em apenas oito pedidos, 0,96% do total. A base não mostra combinações fortes o bastante para sugerir venda casada.

Cuidado: Cada par é contado uma vez por pedido, com o menor ID primeiro. O denominador inclui pedidos de um único produto. Frequência conjunta não demonstra efeito causal.

13.13 Q13/P13 — Intervalo até envio por transportadora

Pergunta: Qual é o intervalo até o envio registrado por transportadora?

Cálculo: Subtrair a data do pedido da data de envio onde ela existe; contar ausências separadamente e preservar as transportadoras sem pedidos.

Recursos: SQL: Diferença de datas, AVG e FILTER. MongoDB: \$lookup, \$dateDiff e contagens condicionais.

Resultado: 6 linhas; todas exibidas.

transportadora (ID)	transportadora	pedidos	com data de envio	sem data de envio
1	Speedy Express	249	245	4
2	United Package	326	315	11
3	Federal Shipping	255	249	6
4	Alliance Shippers	0	0	0
5	UPS	0	0	0
6	DHL	0	0	0

transportadora (ID)	soma de dias	média (dias)	após data requerida
1	2100	8,57	12
2	2909	9,23	16
3	1861	7,47	9
4	0	—	0
5	0	—	0
6	0	—	0

Leitura: Speedy Express leva em média 8,57 dias até o envio, sobre 245 envios registrados; quatro de seus 249 pedidos não têm data. No total, há 21 datas ausentes e 37 envios registrados depois da data requerida.

Cuidado: O intervalo vai até o envio registrado e não mede a entrega ao cliente. Datas ausentes não entram na média como zero nem provam que o pedido não foi enviado.

13.14 Q14/P14 — Descontos sobre o valor dos pedidos

Pergunta: Quanto os descontos reduzem o valor bruto dos itens?

Cálculo: Somar preço vezes quantidade, somar desconto monetário e subtrair para obter valor após descontos; dividir desconto pelo bruto para a taxa ponderada.

Recursos: SQL: Expressões numeric, SUM e razão de somas. MongoDB: Expressões Decimal128, \$group e \$divide.

Resultado: 8 linhas; todas exibidas.

categoria (ID)	valor bruto	desconto	valor	desconto (%)
1	286.526,95	18.658,77	267.868,18	6,51
2	113.694,75	7.647,67	106.047,09	6,73
3	177.099,10	9.741,88	167.357,23	5,50
4	251.330,50	16.823,22	234.507,29	6,69
5	100.726,80	4.982,21	95.744,59	4,95
6	178.188,80	15.166,44	163.022,36	8,51
7	105.268,60	5.284,02	99.984,58	5,02
8	141.623,09	10.361,35	131.261,74	7,32

Leitura: Em Beverages, o desconto soma 18.658,77 sobre bruto de 286.526,95, aproximadamente 6,51%. O cálculo mede redução registrada, sem estimar efeito do desconto sobre a demanda.

Cuidado: A taxa é ponderada pelo valor bruto, diferente da média simples das taxas dos itens. Sem custos, o desconto não pode ser convertido em perda de lucro.

13.15 Q15/P15 — Vendas próprias e da equipe

Pergunta: Quanto foi registrado por cada funcionário e sua equipe?

Cálculo: Percorrer os subordinados, incluir o próprio responsável e somar as vendas de cada membro uma vez dentro da equipe.

Recursos: SQL: WITH RECURSIVE e pré-agregação. MongoDB: \$graphLookup, \$setUnion e \$lookup.

Resultado: 9 linhas; todas exibidas.

gestor (ID)	membros	valor próprio	pedidos da equipe	valor da equipe
1	1	192.107,60	123	192.107,60
2	9	166.537,76	830	1.265.793,04
3	1	202.812,84	127	202.812,84
4	1	232.890,85	156	232.890,85
5	4	68.792,28	224	344.581,71
6	1	73.913,13	67	73.913,13
7	1	124.568,24	72	124.568,24
8	1	126.862,28	104	126.862,28
9	1	77.308,07	43	77.308,07

Leitura: Andrew Fuller (gestor 2) está no topo da hierarquia: sua equipe reúne os nove funcionários e os 830 pedidos. Seu valor próprio é 166.537,76. Steven Buchanan (gestor 5) lidera uma equipe de quatro pessoas, com 344.581,71.

Cuidado: As equipes se sobrepõem na hierarquia; somar as linhas de todos os gestores contaria pedidos mais de uma vez. A equipe hierárquica não corresponde à divisão por território.

13.16 Q16/P16 — Distribuição trimestral por ano e categoria

Pergunta: Como o valor se distribui por ano, trimestre e categoria?

Cálculo: Produzir totais trimestrais e sua decomposição por categoria; marcar trimestres cuja cobertura ultrapassa as datas disponíveis.

Recursos: SQL: GROUPING SETS e GROUPING. MongoDB: \$facet, \$concatArrays e agregação temporal.

Resultado: 72 linhas; exibidos os oito totais trimestrais.

ano	trimestre	categoria (ID)	valor	nível	parcial
1996	3	—	79.728,57	total	sim
1996	4	—	128.355,40	total	não
1997	1	—	138.288,93	total	não
1997	2	—	143.177,05	total	não
1997	3	—	153.937,77	total	não
1997	4	—	181.681,46	total	não
1998	1	—	298.491,55	total	não
1998	2	—	142.132,31	total	sim

Leitura: O primeiro trimestre de 1998 foi o maior da série (298.491,55), mais que o dobro do mesmo trimestre de 1997 (138.288,93). O terceiro trimestre de 1996 e o segundo de 1998 têm cobertura parcial.

Cuidado: Totais e categorias estão em níveis diferentes e não devem ser somados juntos. Com menos de dois anos completos, a série não permite afirmar sazonalidade recorrente.

14. Recursos próprios do MongoDB e MapReduce

Além dos 16 pipelines, três consultas mostram recursos que dependem da estrutura em documento.

Condições no mesmo item (\$elemMatch). A pergunta é: quais pedidos têm um item do produto 11, com pelo menos 10 unidades e algum desconto? Com \$elemMatch, as três condições precisam valer para o mesmo item do array. Escritas separadamente, cada condição pode ser atendida por um item diferente do mesmo pedido.

Valor do pedido sem desmontá-lo (\$map e \$reduce). O \$reduce soma os itens dentro do documento e o \$map extrai os nomes dos produtos, sem transformar o pedido em várias linhas.

Várias respostas numa consulta (\$facet). Uma única consulta sobre os pedidos de 1997 devolve o volume, a situação da data de envio e a distribuição por quantidade de itens.

Com \$elemMatch, 11 pedidos têm um item que atende às três condições ao mesmo tempo. Com as mesmas condições escritas separadamente sobre o array, a contagem sobe para 16: nos 5 pedidos a mais, as condições são satisfeitas por itens diferentes.

O pedido 10248 contém Queso Cabrales, Singaporean Hokkien Fried Mee, Mozzarella di Giovanni; o \$reduce somou 440,00 sem expandir os itens. O painel de 1997 devolveu, em um único documento, 408 pedidos, a situação da data de envio e a distribuição por quantidade de itens:

Faceta	Resultado
volume	408 pedidos
envio	com data: 408
tamanho	1 item: 62; 2 itens: 139; 3 itens: 127; 4 itens: 63; 5 itens: 16; 6 itens: 1

14.1 MapReduce

O MapReduce somou a quantidade vendida de cada produto. A função map emite o par (produto, quantidade) para cada item, e a função reduce soma as quantidades de cada produto. O resultado foi comparado automaticamente com um pipeline de \$unwind e \$group: **77 produtos, 51.317 unidades e todos os totais iguais.**

```
map: function () {
  this.items.forEach(function (i) { emit(i.product._id, i.quantity); });
},
reduce: function (produto, quantidades) { return Array.sum(quantidades); }
```

O MapReduce está depreciado desde o MongoDB 5.0, e a documentação recomenda pipelines para novos desenvolvimentos [7]. Ele foi implementado porque a disciplina o pede, e a comparação mostra que o pipeline equivalente é mais curto e mais legível. A demonstração usa só quantidades inteiras, para não passar dinheiro por números de ponto flutuante do JavaScript.

15. Comparação de sintaxe e complexidade

SQL descreve o resultado desejado: quais tabelas juntar, como agrupar, o que filtrar. Um pipeline descreve uma sequência de etapas pelas quais os documentos passam. Nos dois casos, um otimizador decide a execução, e a ordem escrita não é necessariamente a ordem executada.

A tabela resume os recursos de cada par. As linhas de SQL (sem comentários) e os estágios do pipeline (sem contar os internos de \$lookup e \$facet) indicam tamanho, não dificuldade.

Par	SQL: recursos	Linhas SQL	MongoDB: recursos	Estágios
01	JOIN, GROUP BY e SUM OVER	8	\$unwind, \$group e \$setWindowFields	6
02	CTEs, generate_series e agregação em dois níveis	10	\$map, \$group, \$facet e calendário por \$dateAdd	8
03	NOT EXISTS e junções cadastrais	7	\$lookup com \$limit e filtro de array vazio	4
04	LEFT JOIN, agregação e janela global	10	\$lookup, \$group e \$setWindowFields	5
05	Comparação entre colunas, FILTER e array_agg	7	\$expr, \$set, \$group e \$push	6
06	LEFT JOIN, MAX e comparação de datas	9	\$lookup, \$dateDiff e tratamento de null	5
07	ROW_NUMBER com PARTITION BY	7	\$setWindowFields e soma cumulativa de 1	7
08	SUM OVER com ROWS UNBOUNDED PRECEDING	10	\$setWindowFields com janela até current	9
09	LAG e NULLIF	13	\$shift e \$cond	10
10	CTEs, janela cumulativa e CASE	13	\$lookup, duas janelas e \$switch	5
11	CTE e CASE	11	\$lookup, \$dateDiff e condições	5
12	Autojunção de itens e agregação	8	Expansão de arrays, \$match e \$group	6
13	Diferença de datas, AVG e FILTER	7	\$lookup, \$dateDiff e contagens condicionais	4
14	Expressões numeric, SUM e razão de somas	4	Expressões Decimal128, \$group e \$divide	6
15	WITH RECURSIVE e pré-agregação	15	\$graphLookup, \$setUnion e \$lookup	5
16	GROUPING SETS e GROUPING	11	\$facet, \$concatArrays e agregação temporal	11

15.1 Onde cada modelo foi mais natural

O documento ajudou quando a pergunta é sobre o pedido. Valor por pedido, ticket mensal e pares de produtos usam os itens que já estão dentro do documento: basta \$map ou \$unwind, sem junção.

O SQL foi mais direto quando a pergunta parte de um cadastro ou cruza entidades. Clientes sem compra, vendas por funcionário e por transportadora começam por uma coleção de cadastro e precisam de \$lookup para chegar aos pedidos. Em SQL, é um LEFT JOIN. A hierarquia de funcionários usa WITH RECURSIVE no PostgreSQL e \$graphLookup no MongoDB, com tamanho parecido.

Ranking e janelas existem nos dois. ROW_NUMBER e SUM OVER têm equivalente em \$setWindowFields. Houve uma diferença prática: no MongoDB 7, \$documentNumber aceita um único campo de ordenação [8], então o ranking com desempate por ID foi feito com uma soma acumulada de 1.

Montar um calendário completo é mais longo no MongoDB. No SQL, generate_series cria os 23 meses e um LEFT JOIN preenche os vazios. No pipeline, foi preciso gerar a lista com \$range e \$dateAdd e procurar cada mês com \$filter. Por isso P02, P08 e P09 estão entre os pipelines com mais estágios.

Em resumo: a incorporação dos itens simplificou as consultas centradas no pedido, mas não eliminou as junções. Elas apenas mudaram de lugar.

16. Performance e otimização

16.1 Como foi medido

As 16 consultas e os 16 pipelines rodaram na mesma máquina, sobre os mesmos dados, por conexões já abertas (psycopg e PyMongo). O tempo vai do envio da consulta até o consumo de **todas** as linhas pelo programa. Cada par teve cinco execuções de aquecimento e vinte medições, alternando qual banco rodava primeiro. O relatório usa a **mediana** e o intervalo entre o primeiro e o terceiro quartil (Q1–Q3), que mostra a variação entre as medições.

Ambiente: PostgreSQL 16.15 e MongoDB 7.0.40 em Docker, no WSL2, processador Intel(R) Core(TM) i7-14700HX e 7,8 GB de memória, sem limite de CPU ou memória para os containers. Os planos de execução (EXPLAIN e explain) foram coletados à parte, para não interferir nos tempos.

16.2 Tempos das 16 análises

Par	PostgreSQL: mediana (ms)	PostgreSQL: Q1–Q3	MongoDB: mediana (ms)	MongoDB: Q1–Q3
01	1,011	0,991–1,039	2,281	2,222–2,304
02	1,391	1,321–1,421	2,475	2,319–2,651
03	0,503	0,482–0,533	2,846	2,774–2,967
04	1,340	1,306–1,420	3,916	3,748–3,966
05	0,218	0,213–0,231	0,502	0,491–0,545
06	0,426	0,384–0,469	6,312	5,879–6,645
07	1,162	1,130–1,199	2,711	2,629–2,787
08	1,380	1,326–1,414	2,475	2,396–2,573
09	1,389	1,342–1,455	2,520	2,416–2,653
10	1,594	1,560–1,625	6,597	6,473–6,842
11	1,504	1,457–1,591	6,771	6,576–7,064
12	2,872	2,800–2,960	8,242	7,925–8,691
13	0,375	0,356–0,396	2,411	2,256–2,462
14	1,441	1,419–1,483	3,644	3,561–3,739
15	1,258	1,229–1,292	5,706	5,605–5,975
16	1,480	1,429–1,513	4,443	4,298–4,567

O PostgreSQL teve a menor mediana nos 16 pares, com o MongoDB entre 1,8 e 14,8 vezes mais lento. Os intervalos Q1–Q3 não se sobrepõem em nenhum par, então a diferença é consistente nesta execução. Ela vale para este volume, esta máquina e estes índices.

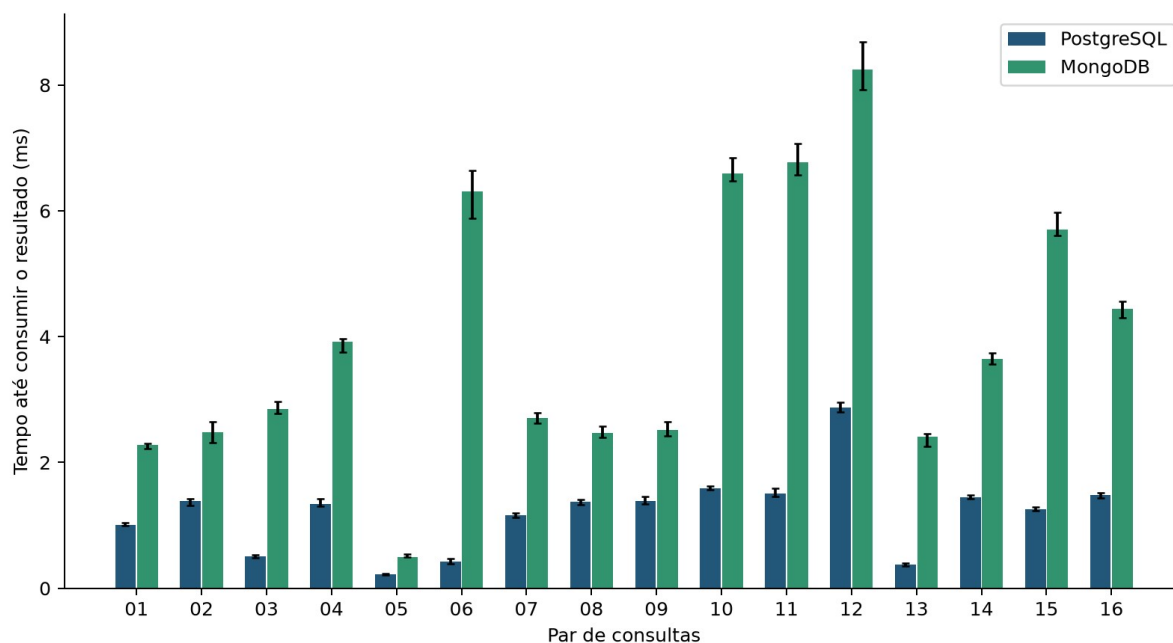


Figura 5 — Mediana das 20 medições por par, em milissegundos; as barras indicam Q1 e Q3.

As maiores diferenças relativas estão em P06 (14,8×), P13 (6,4×), P03 (5,7×), P15 (4,5×), P11 (4,5×), P10 (4,1×). Todos esses pipelines começam por um cadastro (produtos, clientes, funcionários ou transportadoras) e fazem um \$lookup nos pedidos para cada documento, enquanto o PostgreSQL resolve a junção de uma vez. Os planos mostraram ainda que o MongoDB não tinha índice em `employee_id` nem em `shipper_id`: cada \$lookup de P04 e P15 examinava os 830 pedidos.

16.3 Teste da Entrega 4: os índices que faltavam

Para saber quanto da diferença vinha dos índices ausentes, os três pipelines afetados foram medidos em duas cópias isoladas do MongoDB: com os índices atuais e com índices em `employee_id` e `shipper_id`. O PostgreSQL foi medido nas mesmas rodadas. Os resultados foram iguais nas três versões.

Par	PostgreSQL (ms)	MongoDB, índices atuais (ms)	MongoDB com os novos índices (ms)	Redução no MongoDB
04	1,506	3,941	2,867	27,3%
13	0,416	2,341	1,793	23,4%
15	1,364	5,855	5,330	9,0%

Par	Documentos examinados, índices atuais	Documentos examinados, com os novos índices
04	7.470	830
13	4.980	830
15	7.470	1.746

Os índices reduziram o trabalho e o tempo, com intervalos Q1–Q3 sem sobreposição. Mesmo assim, o PostgreSQL continuou entre 1,9 e 4,3 vezes mais rápido. O índice explica parte da diferença; o resto vem da estratégia de um \$lookup por documento e da conversão dos documentos pelo driver. Os índices foram testados só nas cópias; o banco original não foi alterado.

16.4 Estudos de otimização

Quatro consultas foram reescritas ou receberam índices em cópias isoladas dos dados. Em cada estudo, todas as versões rodaram nas mesmas rodadas, em ordem alternada, e devolveram o mesmo resultado. Quando os intervalos Q1–Q3 se sobrepõem, a diferença é tratada como inconclusiva.

Estudo	Versão testada	PostgreSQL: mediana (Q1–Q3), ms	MongoDB: mediana (Q1–Q3), ms
C01	original	0,470 (0,452–0,492)	1,007 (0,958–1,062)

Estudo	Versão testada	PostgreSQL: mediana (Q1–Q3), ms	MongoDB: mediana (Q1–Q3), ms
C01	reescrita	0,364 (0,347–0,385)	0,684 (0,676–0,731)
C01	com_indice	0,322 (0,310–0,347)	0,507 (0,488–0,526)
C02	sem_composto	0,199 (0,183–0,206)	0,644 (0,632–0,664)
C02	com_composto	0,173 (0,160–0,192)	0,441 (0,429–0,477)
C03	original	3,353 (3,293–3,443)	8,300 (8,062–8,710)
C03	reescrita	2,958 (2,871–3,059)	8,137 (7,921–8,592)
C04	original	1,558 (1,470–1,630)	6,440 (6,095–6,794)
C04	reescrita	1,301 (1,261–1,355)	5,856 (5,700–6,007)

C01 — Filtro mensal. A versão original calcula o mês de cada pedido com `to_char` (no MongoDB, `$dateToString`) e compara o texto; a reescrita filtra o intervalo de datas direto; a terceira versão acrescenta um índice em `order_date`. Redução da mediana — Reescrita do filtro: PostgreSQL 22,4%, MongoDB 32,1%; Índice por data: PostgreSQL 11,5%, MongoDB 25,9%.

C02 — Cliente e período. Pedidos de VINET em 1996, sem e com índice composto (`customer_id`, `order_date`). Redução da mediana — Índice composto: PostgreSQL 13,1% (inconclusivo), MongoDB 31,5%.

C03 — Pares de produtos. A versão original em SQL usa uma CTE `MATERIALIZED` que gera todos os pares antes de filtrar `A < B`; sem essa instrução, o próprio PostgreSQL já aplicaria o filtro na junção. No MongoDB, a original carrega o documento inteiro e só descarta os pares repetidos no fim. Redução da mediana — Reescrita: PostgreSQL 11,8%, MongoDB 2,0% (inconclusivo).

C04 — Vendas da equipe. A versão original junta a hierarquia com cada pedido e só depois soma; a reescrita soma os pedidos por funcionário antes de juntar. Redução da mediana — Reescrita: PostgreSQL 16,5%, MongoDB 9,1%.

Os resultados confirmam o que se esperava de cada mudança: filtrar pela coluna, e não por uma função dela, e indexar o acesso mais seletivo trouxeram os maiores ganhos; reduzir resultados intermediários ajudou no PostgreSQL e teve efeito pequeno ou inconclusivo no MongoDB.

16.5 Limites

Os dados cabem em memória e as consultas levam poucos milissegundos, então parte do tempo medido é comunicação e conversão de tipos pelo driver. Não foram medidos concorrência, escrita, grandes volumes nem distribuição em vários servidores. Durante as medições, outros containers sem relação com o projeto estavam ativos na máquina, o que pode acrescentar variação, mas afeta os dois bancos igualmente, porque as execuções foram intercaladas.

17. Guia de decisão: quando usar cada tecnologia

A comparação não aponta um vencedor geral. Ela mostra que cada modelo resolve melhor um tipo de problema. O quadro abaixo resume os critérios, sempre com a evidência que o projeto produziu.

Critério	PostgreSQL	MongoDB	O que o projeto mostrou
Integridade entre entidades	Chaves estrangeiras, CHECK e UNIQUE garantidos pelo banco	Validador só confere o formato do documento	No MongoDB, referências foram conferidas por consulta depois da carga (cap. 8)
Ler um pedido completo	Junção entre orders e order_items	Um único documento	O pedido 10248 é 4 linhas em 2 tabelas ou 1 documento (cap. 7)
Perguntas que partem de cadastros	LEFT JOIN direto	\$lookup por documento	P06, P10, P11 e P15 estão entre as maiores diferenças de tempo (cap. 16)
Relatórios que cruzam várias entidades	Natural, com junções e janelas	Possível, com pipelines mais longos	Os 16 pares foram equivalentes, mas os de calendário e cadastro ficaram mais longos no MongoDB (cap. 15)
Mudança em dados de cadastro	Um único lugar para alterar	Cópias nos documentos precisam ser atualizadas	Nome e categoria do produto estão copiados em cada item (cap. 7)
Estrutura variável entre registros	Exige colunas ou tabelas novas	Cada documento pode ter campos próprios	Não exercitado: a Northwind tem estrutura uniforme
Precisão de valores monetários	numeric	Decimal128	Os dois deram o mesmo total exato; o risco está no arredondamento, não no banco (cap. 4)
Desempenho neste volume	Mediana menor nos 16 pares	1,8 a 14,8 vezes mais lento	Diferença real, mas parte dela vinha de índices ausentes (cap. 16)

17.1 Quando escolher cada um

PostgreSQL é a escolha para o **sistema de registro**: onde as vendas são gravadas e precisam respeitar regras, onde várias entidades se relacionam e onde surgem perguntas novas a cada dia. A Northwind é um caso típico: dados estruturados, relacionamentos fortes e relatórios que cruzam clientes, produtos, funcionários e datas.

MongoDB faz sentido quando o acesso principal é **ler ou gravar um agregado inteiro**, como um pedido com seus itens numa tela de “meus pedidos”, um catálogo cujos produtos têm atributos diferentes entre si ou um registro de eventos com formato variável. Nesses casos, um documento evita juntar peças a cada leitura.

A **arquitetura híbrida** se justifica quando os dois padrões convivem em volume: o relacional registra e garante as regras; o documental serve leituras específicas. O preço é manter a extração, as cópias e dois bancos em operação.

17.2 Recomendação para a Northwind

Para o volume e as perguntas deste projeto, o **PostgreSQL sozinho atende** com folga: respondeu às 16 perguntas mais rápido, garante a integridade sem código extra e não exige sincronização. A camada MongoDB passaria a valer o custo se a empresa criasse, por exemplo, um portal em que muitos clientes consultam o histórico completo de pedidos. Nesse caso, a arquitetura deste projeto, com o PostgreSQL como fonte e o MongoDB como cópia de leitura, já está pronta e validada.

18. Escalabilidade e manutenção

Este capítulo é uma análise argumentada. Com 830 pedidos, todos os dados cabem em memória, e nenhum teste de grande volume, concorrência ou distribuição foi feito. As medições do capítulo 16 não devem ser projetadas para milhões de pedidos.

18.1 Como cada modelo cresceria

Aspecto	PostgreSQL	MongoDB
Crescimento principal	Vertical (servidor maior) e réplicas de leitura	Horizontal, distribuindo coleções entre servidores (sharding)
Tabela ou coleção que mais cresce	orders e order_items; candidatas a particionamento por data	orders; a chave de distribuição precisaria seguir o acesso, como customer_id
Ponto de atenção	Junções sobre tabelas grandes dependem de índices e estatísticas	\$lookup entre coleções distribuídas fica mais caro; o modelo precisa evitar junções nas consultas frequentes
Tamanho do documento	Não se aplica	Há um limite por documento; um pedido com dezenas de itens fica muito longe dele, mas um cliente com todos os seus pedidos dentro não ficaria

O próprio projeto deu um sinal do que acontece com volume maior. Sem índice em `employee_id`, cada \$lookup por funcionário examinou os 830 pedidos (7.470 documentos no total). Com o índice, foram 830. A diferença de tempo hoje é de cerca de 1 milissegundo; com milhões de pedidos, a mesma falta de índice faria a consulta varrer a coleção inteira para cada funcionário.

18.2 Custos de manutenção observados

Mudanças de cadastro. No PostgreSQL, renomear um produto altera uma linha. No MongoDB, o nome está copiado em todos os itens daquele produto, em todos os pedidos, e precisa ser atualizado também.

Mudanças de estrutura. Acrescentar um campo é mais simples no MongoDB, porque documentos antigos e novos convivem. Em compensação, o validador e as consultas precisam acompanhar a mudança, ou a flexibilidade vira inconsistência.

Índices. Nos dois bancos, índices precisam ser revistos conforme as consultas mudam. O teste da Entrega 4 mostrou que dois índices esquecidos explicavam até 27% do tempo de algumas consultas no MongoDB.

Dois bancos em operação. A arquitetura híbrida dobra backups, monitoramento e atualizações, e acrescenta a extração, que precisa ser executada e conferida sempre que a fonte mudar. A conferência automática construída aqui (reconstrução das 3.311 linhas) é o que torna essa manutenção segura.

18.3 Custo e benefício das decisões técnicas

Decisão	Custo	Benefício
Decimais exatos e arredondamento só no fim	Cuidado em cada cálculo	Totais idênticos nos dois bancos; evitou a diferença de 0,25
Schema próprio (nw) separado da fonte	Um script de carga a mais	Regras de domínio no banco e a fonte preservada para comparação
Itens incorporados no pedido	Cópias de nome e categoria a manter	Pedido lido sem junção; consultas de pedido mais simples
Conferência em três caminhos	Tempo de desenvolvimento	Confiança de que SQL e MongoDB não erraram juntos
Arquitetura híbrida	Dois bancos e uma extração	Comparação justa; pronta para uma camada de leitura, se necessária

19. Conclusões

O projeto implementou a mesma base de vendas em PostgreSQL e MongoDB, provou que a migração preservou todos os dados e respondeu a 16 perguntas de negócio nos dois bancos, com resultados idênticos e conferidos por um terceiro cálculo.

19.1 O que os dados mostram

- **A receita registrada é concentrada.** Beverages responde por 21,16% do valor, e um único produto, o Côte de Blaye, por 11,17% de tudo o que foi vendido no período.
- **Poucos clientes sustentam a maior parte do valor.** 34 dos 89 clientes com compras formam a classe A da curva ABC, com 80% do valor.
- **Há um cliente importante parado.** A Mère Paillarde, da classe A, não compra desde 30/10/1997, mais de seis meses antes do fim da base. Cruzar a curva ABC com a análise de inatividade aponta onde agir primeiro.
- **O valor mensal cresceu ao longo de 1997 e no início de 1998**, com o pico em abril de 1998. A queda em maio de 1998 é efeito da base, que termina no dia 6, e não uma retração.
- **O estoque tem 18 produtos abaixo do ponto de reposição**, dois deles sem cobertura nem com o que já foi encomendado. Um dos 18 está descontinuado, o que mostra que o indicador precisa ser lido junto com o status do produto.

Todas essas conclusões tratam de valor registrado. Sem custos, a base não permite falar em lucro, e sem histórico longo, não permite afirmar sazonalidade.

19.2 O que a comparação mostra

- **Os dois modelos chegam às mesmas respostas**, desde que as regras de cálculo sejam fixadas antes, principalmente o arredondamento.
- **O documento simplifica o que gira em torno do pedido**; o relacional é mais direto quando a pergunta parte de um cadastro ou cruza entidades.
- **O PostgreSQL foi mais rápido nos 16 pares** neste volume e nesta máquina. Parte da diferença vinha de índices ausentes no MongoDB, mas, mesmo com eles, o PostgreSQL continuou de 1,9 a 4,3 vezes mais rápido nos pares testados.
- **A escolha depende do padrão de acesso.** Para a Northwind, o PostgreSQL atende sozinho; o MongoDB passa a compensar quando há muitas leituras de pedidos completos ou dados de formato variável.

19.3 Limites

O volume é pequeno, as medições foram feitas numa única máquina, sem concorrência nem teste de escala, e o MongoDB rodou em um único servidor. A base não tem custos, pagamentos nem datas de entrega. Esses limites não invalidam as conclusões, mas definem até onde elas valem.

20. Como reproduzir

O projeto roda com Docker e o gerenciador uv para Python. Os comandos abaixo são executados na raiz do repositório.

```
# 1. Bancos; a base original carrega sozinha no primeiro uso
docker compose up -d
PSQL="docker compose exec -T postgres psql -v ON_ERROR_STOP=1 -U pi -d northwind"
MONGO="docker compose exec -T mongo mongosh -u pi -p pi --authenticationDatabase admin --quiet"

# 2. Modelo relacional, índices, views e procedures
$PSQL -f /sql/10_ddl.sql
$PSQL -f /sql/20_load.sql
$PSQL -f /sql/30_indexes.sql
$PSQL -f /sql/entrega03/00_views.sql
$PSQL -f /sql/entrega03/10_procedures.sql

# 3. Documentos: extração em SQL, carga e índices no MongoDB
docker compose exec -T postgres psql -X -qAt -v ON_ERROR_STOP=1 -U pi -d northwind \
  -f /sql/entrega02/01_exportar_documentos.sql > mongo/entrega02/dados/documentos.ejson
$MONGO --file /mongo/entrega02/02_carregar.js
$MONGO --file /mongo/entrega02/03_indices.js

# 4. Conferências e medições
uv sync
uv run python etl/validar_analytics.py
uv run python etl/conferir_python.py
(cd etl && uv run python testar_analytics.py)
uv run python bench/entrega03.py
uv run python bench/entrega04.py
```

Cada consulta também pode ser executada sozinha: os arquivos sql/queries/QNN.sql no psql ou em qualquer cliente SQL, e os arquivos mongo/pipelines/PNN.js no mongosh.

Referências

- [1] IBM. **CRISP-DM Help Overview**. IBM SPSS Modeler. Disponível em: <https://www.ibm.com/docs/en/spss-modeler/18.6.0?topic=dm-crisp-help-overview>. Acesso em: 5 set. 2026.
- [2] PTHOM. **Northwind sample database for PostgreSQL**. Disponível em: https://github.com/pthom/northwind_psql. Acesso em: 5 set. 2026.
- [3] IPOG. **Livro Digital — Projeto Integrador**. Unidade 4: Projeto Área 03 — Banco de Dados, seções 4.5.2 a 4.5.7. Material didático da disciplina.
- [4] POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation: Numeric Types**. Disponível em: <https://www.postgresql.org/docs/16/datatype-numeric.html>. Acesso em: 5 set. 2026.
- [5] MONGODB. **Best Practices for Data Modeling in MongoDB**. Disponível em: <https://www.mongodb.com/docs/manual/data-modeling/best-practices/>. Acesso em: 5 set. 2026.
- [6] MONGODB. **Model Monetary Data**. Disponível em: <https://www.mongodb.com/docs/manual/tutorial/model-monetary-data/>. Acesso em: 5 set. 2026.
- [7] MONGODB. **MongoDB 7.0: Map-Reduce**. Disponível em: <https://www.mongodb.com/docs/v7.0/core/map-reduce/>. Acesso em: 22 set. 2026.
- [8] MONGODB. **MongoDB 7.0: \$documentNumber**. Disponível em: <https://www.mongodb.com/docs/v7.0/reference/operator/aggregation/documentnumber/>. Acesso em: 22 set. 2026.

Apêndice A — Consultas SQL e pipelines completos

Os códigos abaixo são os mesmos arquivos executados na validação. Cada consulta SQL pode ser executada no PostgreSQL e cada pipeline, no mongosh; o trecho final de cada arquivo .js apenas imprime o resultado. Datas e regras seguem o capítulo 11.

A.1 Par Q01/P01

PostgreSQL:

```
-- Q01 – Valor e participação por categoria
-- Pergunta: Quais categorias concentram o valor dos pedidos?
-- Resultado: Uma linha por categoria com vendas: unidades, valor após descontos e fatia (%) do
valor total.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH totais AS (
  SELECT category_id, sum(quantity) unidades, sum(valor) valor
  FROM nw_analytics.vw_item_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01' GROUP BY category_id
)
SELECT t.category_id, c.category_name categoria, t.unidades, t.valor,
100*t.valor/nullif(sum(t.valor) OVER (), 0) participacao
FROM totais t JOIN nw.categories c USING(category_id)
ORDER BY t.category_id;
MongoDB:
```

```
// P01 – Valor e participação por categoria
// Pergunta: Quais categorias concentram o valor dos pedidos?
// Resultado: Uma linha por categoria com vendas: unidades, valor após descontos e fatia (%) do
valor total.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: { $gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    { $unwind: "$items" },
    {
      $group: {
        _id: "$items.product.category._id",
        categoria: { $first: "$items.product.category.name" },
        unidades: { $sum: "$items.quantity" },
        valor: {
          $sum: {
            $multiply: [
              "$items.unit_price",
              "$items.quantity",
              { $subtract: [Decimal128("1"), "$items.discount"]}
            ]
          }
        }
      }
    },
    {
      $setWindowFields: { output: { total: { $sum: "$valor", window: { documents: ["unbounded", "unbounded"] } } } }
    },
    {
      $project: {
        _id: 0,
        category_id: "$_id",
        categoria: 1,
        unidades: 1,
        valor: 1,
        participacao: {
          $cond: [
            { $eq: ["$total", 0] },
            null,
            { $divide: [{ $multiply: ["$valor", Decimal128("100")] }, "$total"] }
          ]
        }
      }
    },
    { $sort: { category_id: 1 } }
  ]
};
```

```
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.2 Par Q02/P02

PostgreSQL:

```
-- Q02 – Pedidos e ticket médio mensal
-- Pergunta: Como variam o número de pedidos e o ticket médio mensal?
-- Resultado: Uma linha por mês, inclusive sem pedidos: quantidade de pedidos, valor e valor
médio por pedido.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH calendario AS (
  SELECT generate_series(DATE '1996-07-01', DATE '1998-06-01'-INTERVAL '1 month', INTERVAL '1
month')::date mes
), totais AS (
  SELECT date_trunc('month',order_date)::date mes, count(*) pedidos, sum(valor) valor
  FROM nw_analytics.vw_pedido_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01' GROUP BY 1
), mensal AS (
  SELECT c.mes,coalesce(t.pedidos,0) pedidos,coalesce(t.valor,0) valor
  FROM calendario c LEFT JOIN totais t USING(mes)
)
SELECT mes,pedidos,valor,valor/nullif(pedidos,0) ticket FROM mensal ORDER BY mes;
```

MongoDB:

```
// P02 – Pedidos e ticket médio mensal
// Pergunta: Como variam o número de pedidos e o ticket médio mensal?
// Resultado: Uma linha por mês, inclusive sem pedidos: quantidade de pedidos, valor e valor
médio por pedido.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: { $gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {
      $set: {
        valor: {
          $sum: {
            $map: {
              input: "$items",
              as: "i",
              in: {
                $multiply: [
                  "$$i.unit_price",
                  "$$i.quantity",
                  {$subtract: [Decimal128("1"), "$$i.discount"]}
                ]
              }
            }
          }
        }
      }
    },
    {
      $facet: {
        dados: [
          {
            $group: {
              _id: {$dateTrunc: {date: "$order_date", unit: "month", timezone: "UTC"}},
              pedidos: {$sum: 1},
              valor: {$sum: "$valor"}
            }
          }
        ]
      }
    },
    {
      $project: {
        linhas: {
          $map: {
            input: {
              $range: [
                0,
                {

```

```

        $dateDiff: {
          startDate: ISODate("1996-07-01T00:00:00Z"),
          endDate: ISODate("1998-06-01T00:00:00Z"),
          unit: "month"
        }
      }
    ]
  },
  as: "k",
  in: {
    $let: {
      vars: {
        m: {
          $dateAdd: {
            startDate: ISODate("1996-07-01T00:00:00Z"),
            unit: "month",
            amount: "$$k"
          }
        }
      },
      in: {
        $let: {
          vars: {
            r: {
              $arrayElemAt: [
                {
                  $filter: {input: "$dados", as: "d", cond: {$eq: ["$$d._id", "$$m"]}}
                },
                0
              ]
            }
          },
          in: {
            mes: "$$m",
            pedidos: {$ifNull: ["$$r.pedidos", 0]},
            valor: {$ifNull: ["$$r.valor", Decimal128("0")]}
          }
        }
      }
    }
  }
},
{$unwind: "$linhas"},
{$replaceWith: "$linhas"},
{
  $project: {
    _id: 0,
    mes: {$dateToString: {date: "$mes", format: "%Y-%m-%d", timezone: "UTC"}},
    pedidos: 1,
    valor: 1,
    ticket: {$cond: [{ $eq: ["$pedidos", 0]}, null, {$divide: ["$valor", "$pedidos"]}] }
  },
  {$sort: {mes: 1}}
}
];
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.3 Par Q03/P03

PostgreSQL:

```

-- Q03 – Produtos sem vendas no período
-- Pergunta: Quais produtos não foram vendidos no período?
-- Resultado: Produtos do catálogo sem nenhum item vendido na janela; lista vazia significa que todos venderam.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
SELECT p.product_id,p.product_name produto,c.category_name categoria,s.company_name fornecedor
FROM nw.products p JOIN nw.categories c USING(category_id)
JOIN nw.suppliers s USING(supplier_id)
WHERE NOT EXISTS (
  SELECT 1 FROM nw.order_items i JOIN nw.orders o USING(order_id)
  WHERE i.product_id=p.product_id AND order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01'
) ORDER BY p.product_id;

```

MongoDB:

```
// P03 – Produtos sem vendas no período
// Pergunta: Quais produtos não foram vendidos no período?
// Resultado: Produtos do catálogo sem nenhum item vendido na janela; lista vazia significa que
// todos venderam.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "products",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "items.product._id",
        pipeline: [
          {
            $match: {
              order_date: {
                $gte: ISODate("1996-07-01T00:00:00Z"),
                $lt: ISODate("1998-06-01T00:00:00Z")
              }
            }
          },
          {
            $limit: 1,
            $project: { _id: 0, encontrado: { $literal: true } }
          }
        ],
        as: "vendas"
      }
    },
    {
      $match: { vendas: { $size: 0 } }
    },
    {
      $project: {
        _id: 0,
        product_id: "$_id",
        produto: "$product_name",
        categoria: "$category.name",
        fornecedor: "$supplier.name"
      }
    },
    {
      $sort: { product_id: 1 }
    }
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.4 Par Q04/P04**PostgreSQL:**

```
-- Q04 – Indicadores por funcionário
-- Pergunta: Como se distribuem os pedidos entre os funcionários?
-- Resultado: Uma linha por funcionário, inclusive sem pedidos: pedidos, valor, valor médio por
-- pedido e fatia do total.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH t AS (
  SELECT e.employee_id, e.first_name || ' ' || e.last_name funcionario,
         count(o.order_id) pedidos, coalesce(sum(o.valor), 0) valor
  FROM nw.employees e LEFT JOIN nw_analytics.vw_pedido_valor o
    ON o.employee_id = e.employee_id AND order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01'
  GROUP BY e.employee_id
)
SELECT *, valor/nullif(pedidos, 0) ticket,
       100*valor/nullif(sum(valor) OVER (), 0) participacao
FROM t ORDER BY employee_id;
```

MongoDB:

```
// P04 – Indicadores por funcionário
// Pergunta: Como se distribuem os pedidos entre os funcionários?
// Resultado: Uma linha por funcionário, inclusive sem pedidos: pedidos, valor, valor médio por
// pedido e fatia do total.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "employees",
  pipeline: [
    {
      $lookup: {
        from: "orders",
```

```

        localField: "_id",
        foreignField: "employee_id",
        pipeline: [
            {
                $match: {
                    order_date: {
                        $gte: ISODate("1996-07-01T00:00:00Z"),
                        $lt: ISODate("1998-06-01T00:00:00Z")
                    }
                }
            },
            {
                $group: {
                    _id: null,
                    pedidos: {$sum: 1},
                    valor: {
                        $sum: {
                            $sum: {
                                $map: {
                                    input: "$items",
                                    as: "i",
                                    in: {
                                        $multiply: [
                                            "$$i.unit_price",
                                            "$$i.quantity",
                                            {$subtract: [Decimal128("1"), "$$i.discount"]}
                                        ]
                                    }
                                }
                            }
                        }
                    }
                }
            },
            {
                $as: "vendas"
            },
            {
                $set: {
                    pedidos: {$ifNull: [{ $first: "$vendas.pedidos"}, 0]},
                    valor: {$ifNull: [{ $first: "$vendas.valor"}, Decimal128("0")]}
                }
            },
            {
                $setWindowFields: {output: {total: {$sum: "$valor", window: {documents: ["unbounded", "unbounded"]}}}}
            },
            {
                $project: {
                    _id: 0,
                    employee_id: "$_id",
                    funcionario: {$concat: ["$first_name", " ", "$last_name"]},
                    pedidos: 1,
                    valor: 1,
                    ticket: {$cond: [{ $eq: ["$pedidos", 0]}, null, {$divide: ["$valor", "$pedidos"]}],},
                    participacao: {
                        $cond: [
                            { $eq: ["$total", 0]},
                            null,
                            {$divide: [{ $multiply: ["$valor", Decimal128("100")]}, "$total"]}
                        ]
                    }
                }
            },
            {$sort: {employee_id: 1}}
        ]
    };
    if (typeof ANALYTICS_EXPORT === 'undefined') {
        print(EJSON.stringify(db.getSiblingDB('northwind')
            .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
    }
}

```

A.5 Par Q05/P05

PostgreSQL:

```

-- Q05 – Estoque crítico e cobertura por categoria
-- Pergunta: Quais categorias têm produtos abaixo do ponto de reposição?
-- Resultado: Categorias com produtos abaixo do ponto de reposição, déficit somado e quantos nem
com o encomendado se cobrem.

```

```
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
SELECT p.category_id,c.category_name categoria,count(*) produtos,
       sum(p.reorder_level-p.units_in_stock) deficit,
       count(*) FILTER (WHERE p.units_in_stock+p.units_on_order<p.reorder_level)
       reposicao_insuficiente,
       array_agg(p.product_id ORDER BY p.product_id) produtos_ids
FROM nw.products p JOIN nw.categories c USING(category_id)
WHERE p.units_in_stock<p.reorder_level
GROUP BY p.category_id,c.category_name ORDER BY p.category_id;
MongoDB:
```

```
// P05 – Estoque crítico e cobertura por categoria
// Pergunta: Quais categorias têm produtos abaixo do ponto de reposição?
// Resultado: Categorias com produtos abaixo do ponto de reposição, déficit somado e quantos nem
// com o encomendado se cobrem.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "products",
  pipeline: [
    {
      $match: {
        $expr: {
          $lt: [
            "$units_in_stock",
            "$reorder_level"
          ]
        }
      },
      $set: {
        deficit: {
          $subtract: [
            "$reorder_level",
            "$units_in_stock"
          ]
        },
        insuficiente: {
          $cond: [
            {
              $lt: [
                {
                  $add: [
                    "$units_in_stock",
                    "$units_on_order"
                  ]
                },
                "$reorder_level"
              ]
            },
            1,
            0
          ]
        }
      }
    },
    {
      $sort: {
        _id: 1
      }
    },
    {
      $group: {
        _id: "$category._id",
        categoria: {
          $first: "$category.name"
        },
        produtos: {
          $sum: 1
        },
        deficit: {
          $sum: "$deficit"
        },
        reposicao_insuficiente: {
          $sum: "$insuficiente"
        },
        produtos_ids: {
          $push: "$_id"
        }
      }
    },
    {
      $project: {
        _id: 0,
        category_id: "$_id",
        categoria: 1,
        produtos: 1,
        deficit: 1,
        reposicao_insuficiente: 1,
        produtos_ids: 1
      }
    },
    {
      $sort: {
        category_id: 1
      }
    }
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.6 Par Q06/P06

PostgreSQL:

```
-- Q06 – Clientes inativos e sem compras
-- Pergunta: Quais clientes estão sem compra há mais de seis meses?
-- Resultado: Clientes sem compra desde antes de 06/11/1997 (inativos) e clientes que nunca
-- compraram.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH historico AS (
  SELECT c.customer_id,c.company_name cliente,max(o.order_date) ultima
  FROM nw.customers c LEFT JOIN nw.orders o ON o.customer_id=c.customer_id
  AND o.order_date<=DATE '1998-05-06' GROUP BY c.customer_id
)
SELECT *,DATE '1998-05-06'-ultima recencia,
       CASE WHEN ultima IS NULL THEN 'sem compras' ELSE 'inativo' END situacao
FROM historico WHERE ultima IS NULL OR ultima<DATE '1997-11-06'
ORDER BY customer_id;
```

MongoDB:

```
// P06 – Clientes inativos e sem compras
// Pergunta: Quais clientes estão sem compra há mais de seis meses?
// Resultado: Clientes sem compra desde antes de 06/11/1997 (inativos) e clientes que nunca
compraram.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "customers",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "customer_id",
        pipeline: [
          {
            $match: {order_date: {$lte: ISODate("1998-05-06T00:00:00Z")}},
            {
              $group: {
                _id: null,
                ultima: {$max: "$order_date"},
                frequencia: {$sum: 1},
                valor: {
                  $sum: {
                    $sum: {
                      $map: {
                        input: "$items",
                        as: "i",
                        in: {
                          $multiply: [
                            "$$i.unit_price",
                            "$$i.quantity",
                            {$subtract: [Decimal128("1"), "$$i.discount"]}
                          ]
                        }
                      }
                    }
                  }
                }
              }
            }
          ]
        }
      },
      as: "compras"
    },
    {
      $set: {
        ultima: {$ifNull: [{ $first: "$compras.ultima"}, null]},
        frequencia: {$ifNull: [{ $first: "$compras.frequencia"}, 0]},
        valor: {$ifNull: [{ $first: "$compras.valor"}, Decimal128("0")]}
      }
    },
    {
      $match: {$or: [{ultima: null}, {ultima: {$lt: ISODate("1997-11-06T00:00:00Z")}}]}
    },
    {
      $project: {
        _id: 0,
        customer_id: "$_id",
        cliente: "$company_name",
        ultima: {
          $dateToString: {date: "$ultima", format: "%Y-%m-%d", timezone: "UTC", onNull: null}
        },
        recencia: {
          $dateDiff: {startDate: "$ultima", endDate: ISODate("1998-05-06T00:00:00Z"), unit:
"day"}
        },
        situacao: {$cond: [{ $eq: ["$ultima", null]}, "sem compras", "inativo"]}
      }
    },
    {
      $sort: {customer_id: 1}
    }
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.7 Par Q07/P07

PostgreSQL:

```
-- Q07 – Top cinco produtos por categoria
```

```
-- Pergunta: Quais são os cinco produtos de maior valor em cada categoria?
-- Resultado: Até cinco produtos de maior valor em cada categoria, com posição 1 a 5 e desempate
pelo menor ID.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH t AS (
  SELECT category_id,product_id,sum(valor) valor FROM nw_analytics.vw_item_valor
  WHERE order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01' GROUP BY
  category_id,product_id
), ranking AS (
  SELECT *,row_number() OVER (PARTITION BY category_id ORDER BY valor DESC,product_id) posicao
FROM t
)
SELECT category_id,product_id,valor,posicao FROM ranking WHERE posicao<=5 ORDER BY
category_id,posicao;
MongoDB:
```

```
// P07 – Top cinco produtos por categoria
// Pergunta: Quais são os cinco produtos de maior valor em cada categoria?
// Resultado: Até cinco produtos de maior valor em cada categoria, com posição 1 a 5 e desempate
pelo menor ID.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {$unwind: "$items"},
    {
      $group: {
        _id: {c: "$items.product.category_id", p: "$items.product._id"},
        valor: {
          $sum: {
            $multiply: [
              "$items.unit_price",
              "$items.quantity",
              {$subtract: [Decimal128("1"), "$items.discount"]}
            ]
          }
        }
      }
    },
    {$project: {_id: 0, category_id: "$_id.c", product_id: "$_id.p", valor: 1}},
    {
      $setWindowFields: {
        partitionBy: "$category_id",
        sortBy: {valor: -1, product_id: 1},
        output: {posicao: {$sum: 1, window: {documents: ["unbounded", "current"]}}}
      }
    },
    {$match: {posicao: {$lte: 5}}},
    {$sort: {category_id: 1, posicao: 1}}
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.8 Par Q08/P08

PostgreSQL:

```
-- Q08 – Valor mensal acumulado
-- Pergunta: Qual é o valor acumulado ao longo dos meses?
-- Resultado: Valor de cada mês e o valor acumulado desde julho de 1996; o último acumulado é o
total do período.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH calendario AS (
  SELECT generate_series(DATE '1996-07-01', DATE '1998-06-01'-INTERVAL '1 month', INTERVAL '1
month')::date mes
), totais AS (
  SELECT date_trunc('month',order_date)::date mes, count(*) pedidos, sum(valor) valor
  FROM nw_analytics.vw_pedido_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01' GROUP BY 1
), mensal AS (
  SELECT c.mes,coalesce(t.pedidos,0) pedidos,coalesce(t.valor,0) valor
  FROM calendario c LEFT JOIN totais t USING(mes)
)
```

```
SELECT mes,valor,sum(valor) OVER (ORDER BY mes ROWS UNBOUNDED PRECEDING) acumulado FROM mensal
ORDER BY mes;
```

MongoDB:

```
// P08 – Valor mensal acumulado
// Pergunta: Qual é o valor acumulado ao longo dos meses?
// Resultado: Valor de cada mês e o valor acumulado desde julho de 1996; o último acumulado é o
// total do período.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {
      $set: {
        valor: {
          $sum: {
            $map: {
              input: "$items",
              as: "i",
              in: {
                $multiply: [
                  "$$i.unit_price",
                  "$$i.quantity",
                  {$subtract: [Decimal128("1"), "$$i.discount"]}
                ]
              }
            }
          }
        }
      }
    },
    {
      $facet: {
        dados: [
          {
            $group: {
              _id: {$dateTrunc: {date: "$order_date", unit: "month", timezone: "UTC"}},
              pedidos: {$sum: 1},
              valor: {$sum: "$valor"}
            }
          ]
        ]
      }
    },
    {
      $project: {
        linhas: {
          $map: {
            input: {
              $range: [
                0,
                {
                  $dateDiff: {
                    startDate: ISODate("1996-07-01T00:00:00Z"),
                    endDate: ISODate("1998-06-01T00:00:00Z"),
                    unit: "month"
                  }
                }
              ]
            },
            as: "k",
            in: {
              $let: {
                vars: {
                  m: {
                    $dateAdd: {
                      startDate: ISODate("1996-07-01T00:00:00Z"),
                      unit: "month",
                      amount: "$$k"
                    }
                  }
                },
                in: {
                  $let: {
                    vars: {
                      r: {

```

```
$arrayElemAt: [
    {
        $filter: {input: "$dados", as: "d", cond: {$eq: ["$$d._id", "$$m"]}}
    },
    0
]
},
in: {
    mes: "$$m",
    pedidos: {$ifNull: ["$$r.pedidos", 0]},
    valor: {$ifNull: ["$$r.valor", Decimal128("0")]}
}
}
}
}
}
}
}
}
{
    $setWindowFields: {
        sortBy: {mes: 1},
        output: {acumulado: {$sum: "$valor", window: {documents: ["unbounded", "current"]}}}
    }
},
{
    $project: {
        _id: 0,
        mes: {$dateToString: {date: "$mes", format: "%Y-%m-%d"}},
        valor: 1,
        acumulado: 1
    }
},
{$sort: {mes: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
    print(EJSON.stringify(db.getSiblingDB('northwind')
        .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.9 Par Q09/P09

PostgreSQL:

```
-- Q09 - Variação mensal do valor
-- Pergunta: Quanto o valor mudou em relação ao mês anterior?
-- Resultado: Valor de cada mês comparado ao mês anterior: diferença absoluta e variação percentual.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH calendario AS (
  SELECT generate_series(DATE '1996-07-01', DATE '1998-06-01'-INTERVAL '1 month', INTERVAL '1 month')::date mes
), totais AS (
  SELECT date_trunc('month',order_date)::date mes, count(*) pedidos, sum(valor) valor
  FROM nw_analytics.vw_pedido_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01' GROUP BY 1
), mensal AS (
  SELECT c.mes,coalesce(t.pedidos,0) pedidos,coalesce(t.valor,0) valor
  FROM calendario c LEFT JOIN totais t USING(mes)
), anterior AS (
  SELECT *,lag(valor) OVER (ORDER BY mes) anterior FROM mensal
)
SELECT mes,valor,anterior,valor-anterior variacao,
100*(valor-anterior)/nullif(anterior,0) percentual FROM anterior ORDER BY mes;
```

MongoDB:

```
// P09 – Variação mensal do valor
// Pergunta: Quanto o valor mudou em relação ao mês anterior?
// Resultado: Valor de cada mês comparado ao mês anterior: diferença absoluta e variação percentual.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
```

```

    order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
  },
  {
    $set: {
      valor: {
        $sum: {
          $map: {
            input: "$items",
            as: "i",
            in: {
              $multiply: [
                "$$i.unit_price",
                "$$i.quantity",
                {$subtract: [Decimal128("1"), "$$i.discount"]}
              ]
            }
          }
        }
      }
    }
  },
  {
    $facet: {
      dados: [
        {
          $group: {
            _id: {$dateTrunc: {date: "$order_date", unit: "month", timezone: "UTC"}},
            pedidos: {$sum: 1},
            valor: {$sum: "$valor"}
          }
        }
      ]
    }
  },
  {
    $project: {
      linhas: {
        $map: {
          input: {
            $range: [
              0,
              {
                $dateDiff: {
                  startDate: ISODate("1996-07-01T00:00:00Z"),
                  endDate: ISODate("1998-06-01T00:00:00Z"),
                  unit: "month"
                }
              }
            ]
          },
          as: "k",
          in: {
            $let: {
              vars: {
                m: {
                  $dateAdd: {
                    startDate: ISODate("1996-07-01T00:00:00Z"),
                    unit: "month",
                    amount: "$$k"
                  }
                }
              }
            },
            in: {
              $let: {
                vars: {
                  r: {
                    $arrayElemAt: [
                      {
                        $filter: {input: "$dados", as: "d", cond: {$eq: ["$$d._id", "$$m"]}}
                      },
                      0
                    ]
                  }
                },
                in: {
                  mes: "$$m",
                  pedidos: {$ifNull: ["$$r.pedidos", 0]},
                  valor: {$ifNull: ["$$r.valor", Decimal128("0")]}
                }
              }
            }
          }
        }
      }
    }
  }
}

```

```

    }
  }
}
},
{$unwind: "$linhas"},
{$replaceWith: "$linhas"},
{
  $setWindowFields: {
    sortBy: {mes: 1},
    output: {anterior: {$shift: {output: "$valor", by: -1, default: null}}}
  },
  {$set: {variacao: {$subtract: ["$valor", "$anterior"]}}},
  {
    $project: {
      _id: 0,
      mes: {$dateToString: {date: "$mes", format: "%Y-%m-%d"}},
      valor: 1,
      anterior: 1,
      variacao: 1,
      percentual: {
        $cond: [
          {$eq: ["$anterior", 0]},
          null,
          {$divide: [{$multiply: ["$variacao", Decimal128("100")]}, "$anterior"]}
        ]
      }
    }
  },
  {$sort: {mes: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.10 Par Q10/P10

PostgreSQL:

```

-- Q10 – Curva ABC de clientes
-- Pergunta: Como os clientes se distribuem pela curva ABC de valor?
-- Resultado: Todos os clientes ordenados por valor, com acumulado e classe A/B/C pela
concentração do valor.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH t AS (
  SELECT c.customer_id,c.company_name cliente,count(o.order_id) pedidos,coalesce(sum(o.valor),0)
  valor
  FROM nw.customers c LEFT JOIN nw_analytics.vw_pedido_valor o
  ON o.customer_id=c.customer_id AND order_date >= DATE '1996-07-01' AND order_date < DATE '1998-
06-01' GROUP BY c.customer_id
), a AS (
  SELECT *,sum(valor) OVER () total,
  sum(valor) OVER (ORDER BY valor DESC,customer_id ROWS UNBOUNDED PRECEDING) acumulado FROM t
)
SELECT customer_id,cliente,pedidos,valor,acumulado,
CASE WHEN pedidos=0 THEN 'sem compras'
WHEN acumulado-valor<total*0.80 THEN 'A'
WHEN acumulado-valor<total*0.95 THEN 'B' ELSE 'C' END classe
FROM a ORDER BY valor DESC,customer_id;

```

MongoDB:

```

// P10 – Curva ABC de clientes
// Pergunta: Como os clientes se distribuem pela curva ABC de valor?
// Resultado: Todos os clientes ordenados por valor, com acumulado e classe A/B/C pela
concentração do valor.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "customers",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "customer_id",
        pipeline: [

```

```

    $match: {
      order_date: {
        $gte: ISODate("1996-07-01T00:00:00Z"),
        $lt: ISODate("1998-06-01T00:00:00Z")
      }
    },
  },
  {
    $group: {
      _id: null,
      pedidos: {$sum: 1},
      valor: {
        $sum: {
          $sum: {
            $map: {
              input: "$items",
              as: "i",
              in: {
                $multiply: [
                  "$$i.unit_price",
                  "$$i.quantity",
                  {$subtract: [Decimal128("1"), "$$i.discount"]}
                ]
              }
            }
          }
        }
      }
    }
  }
},
],
as: "compras"
}
},
{
  $set: {
    pedidos: {$ifNull: [{ $first: "$compras.pedidos"}, 0]},
    valor: {$ifNull: [{ $first: "$compras.valor"}, Decimal128("0")]}
  }
},
{
  $setWindowFields: {
    sortBy: {valor: -1, _id: 1},
    output: {
      acumulado: {$sum: "$valor", window: {documents: ["unbounded", "current"]}},
      total: {$sum: "$valor", window: {documents: ["unbounded", "unbounded"]}}
    }
  }
},
{
  $project: {
    _id: 0,
    customer_id: "$_id",
    cliente: "$company_name",
    pedidos: 1,
    valor: 1,
    acumulado: 1,
    classe: {
      $switch: {
        branches: [
          {case: {$eq: ["$pedidos", 0]}, then: "sem compras"},
          {
            case: {
              $lt: [
                {$subtract: ["$acumulado", "$valor"]},
                {$multiply: ["$total", Decimal128("0.80")]}
              ]
            },
            then: "A"
          },
          {
            case: {
              $lt: [
                {$subtract: ["$acumulado", "$valor"]},
                {$multiply: ["$total", Decimal128("0.95")]}
              ]
            },
            then: "B"
          }
        ],
        default: "C"
      }
    }
  }
}

```

```

    }
  }
},
{$sort: {valor: -1, customer_id: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.11 Par Q11/P11

PostgreSQL:

```

-- Q11 – RFM descritivo dos clientes
-- Pergunta: Qual é o perfil de recência, frequência e valor de cada cliente?
-- Resultado: Recência (dias), frequência (pedidos) e valor de cada cliente, com escores 0 a 3 em faixas fixas.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH rfm AS (
  SELECT c.customer_id, DATE '1998-05-06' - max(o.order_date) recencia,
         count(o.order_id) frequencia, coalesce(sum(o.valor), 0) valor
  FROM nw.customers c LEFT JOIN nw_analytics.vw_pedido_valor o
  ON o.customer_id = c.customer_id AND o.order_date <= DATE '1998-05-06'
  GROUP BY c.customer_id
)
SELECT *, CASE WHEN frequencia = 0 THEN 0 WHEN recencia <= 30 THEN 3 WHEN recencia <= 90 THEN 2 ELSE 1
END r,
         CASE WHEN frequencia = 0 THEN 0 WHEN frequencia >= 10 THEN 3 WHEN frequencia >= 5 THEN 2 ELSE 1 END f,
         CASE WHEN frequencia = 0 THEN 0 WHEN valor >= 10000 THEN 3 WHEN valor >= 5000 THEN 2 ELSE 1 END m
FROM rfm ORDER BY customer_id;

```

MongoDB:

```

// P11 – RFM descritivo dos clientes
// Pergunta: Qual é o perfil de recência, frequência e valor de cada cliente?
// Resultado: Recência (dias), frequência (pedidos) e valor de cada cliente, com escores 0 a 3 em faixas fixas.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "customers",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "customer_id",
        pipeline: [
          { $match: { order_date: { $lte: ISODate("1998-05-06T00:00:00Z") } } },
          {
            $group: {
              _id: null,
              ultima: { $max: "$order_date" },
              frequencia: { $sum: 1 },
              valor: {
                $sum: {
                  $sum: {
                    $map: {
                      input: "$items",
                      as: "i",
                      in: {
                        $multiply: [
                          "$$i.unit_price",
                          "$$i.quantity",
                          { $subtract: [Decimal128("1"), "$$i.discount"] }
                        ]
                      }
                    }
                  }
                }
              }
            }
          }
        ]
      },
      as: "compras"
    },
    {
      $set: {
        ultima: { $ifNull: [{ $first: "$compras.ultima" }, null ] },

```

```

        frecuencia: {$ifNull: [{ $first: "$compras.frecuencia"}, 0]},
        valor: {$ifNull: [{ $first: "$compras.valor"}, Decimal128("0")]}
    },
    {
        $set: {
            recencia: {
                $dateDiff: {startDate: "$ultima", endDate: ISODate("1998-05-06T00:00:00Z"), unit:
"day"}
            }
        }
    },
    {
        $project: {
            _id: 0,
            customer_id: "$_id",
            recencia: 1,
            frecuencia: 1,
            valor: 1,
            r: {
                $cond: [
                    {$eq: ["$frecuencia", 0]},
                    0,
                    {
                        $cond: [{ $lte: ["$recencia", 30]}, 3, {$cond: [{ $lte: ["$recencia", 90]}, 2, 1]}
                    ]
                ]
            },
            f: {
                $cond: [
                    {$eq: ["$frecuencia", 0]},
                    0,
                    {
                        $cond: [
                            {$gte: ["$frecuencia", 10]},
                            3,
                            {$cond: [{ $gte: ["$frecuencia", 5]}, 2, 1]}
                        ]
                    }
                ]
            },
            m: {
                $cond: [
                    {$eq: ["$frecuencia", 0]},
                    0,
                    {
                        $cond: [
                            {$gte: ["$valor", Decimal128("10000")]},
                            3,
                            {$cond: [{ $gte: ["$valor", Decimal128("5000")]}, 2, 1]}
                        ]
                    }
                ]
            }
        ]
    },
    {$sort: {customer_id: 1}}
];
if (typeof ANALYTICS_EXPORT === 'undefined') {
    print(EJSON.stringify(db.getSiblingDB('northwind')
        .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.12 Par Q12/P12

PostgreSQL:

```

-- Q12 – Produtos comprados juntos
-- Pergunta: Quais produtos aparecem juntos no mesmo pedido?
-- Resultado: Pares de produtos que aparecem no mesmo pedido, quantos pedidos contêm cada par e o
suporte (%).
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH pares AS (
    SELECT a.product_id produto_a,b.product_id produto_b,count(*) pedidos
    FROM nw.orders o JOIN nw.order_items a USING(order_id)
    JOIN nw.order_items b ON b.order_id=a.order_id AND a.product_id<b.product_id
    WHERE order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01' GROUP BY
a.product_id,b.product_id
), total AS (SELECT count(*) denominador FROM nw.orders WHERE order_date >= DATE '1996-07-01' AND
order_date < DATE '1998-06-01')

```

```
SELECT pares.*,total.denominador,100.0*pedidos/nullif(denominador,0) suporte
FROM pares CROSS JOIN total ORDER BY pedidos DESC,produto_a,produto_b;
```

MongoDB:

```
// P12 – Produtos comprados juntos
// Pergunta: Quais produtos aparecem juntos no mesmo pedido?
// Resultado: Pares de produtos que aparecem no mesmo pedido, quantos pedidos contêm cada par e o
// suporte (%).
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {
      $facet: {
        total: [{$count: "n"}],
        pares: [
          {$project: {_id: 0, ids: "$items.product._id"}},
          {$set: {outros: "$ids"}},
          {$unwind: "$ids"},
          {$unwind: "$outros"},
          {$match: {$expr: {$lt: ["$ids", "$outros"]}}},
          {$group: {_id: {a: "$ids", b: "$outros"}, pedidos: {$sum: 1}}}
        ]
      }
    },
    {
      $set: {denominador: {$ifNull: [{$first: "$total.n"}, 0]}}},
    {
      $unwind: "$pares",
    {
      $project: {
        _id: 0,
        produto_a: "$pares._id.a",
        produto_b: "$pares._id.b",
        pedidos: "$pares.pedidos",
        denominador: 1,
        suporte: {
          $cond: [
            {$eq: ["$denominador", 0]},
            null,
            {
              $divide: [{$multiply: ["$pares.pedidos", Decimal128("100")]}, "$denominador"]
            }
          ]
        }
      }
    }
  ],
  $sort: {pedidos: -1, produto_a: 1, produto_b: 1}
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.13 Par Q13/P13

PostgreSQL:

```
-- Q13 – Intervalo até envio por transportadora
-- Pergunta: Qual é o intervalo até o envio registrado por transportadora?
-- Resultado: Por transportadora: pedidos, datas de envio registradas e ausentes, média de dias
-- até o envio e envios após a data requerida.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
SELECT s.shipper_id,s.company_name transportadora,count(o.order_id) pedidos,
count(o.shipped_date) com_envio,count(o.order_id)-count(o.shipped_date) sem_data,
coalesce(sum(o.shipped_date-o.order_date),0) soma_dias,
avg(o.shipped_date-o.order_date) media_dias,
count(*) FILTER (WHERE o.shipped_date>o.required_date) apos_requerida
FROM nw.shippers s LEFT JOIN nw.orders o ON o.shipper_id=s.shipper_id AND order_date >= DATE
'1996-07-01' AND order_date < DATE '1998-06-01'
GROUP BY s.shipper_id ORDER BY s.shipper_id;
```

MongoDB:

```
// P13 – Intervalo até envio por transportadora
// Pergunta: Qual é o intervalo até o envio registrado por transportadora?
// Resultado: Por transportadora: pedidos, datas de envio registradas e ausentes, média de dias
-- até o envio e envios após a data requerida.
```

```
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "shippers",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "shipper_id",
        pipeline: [
          {
            $match: {
              order_date: {
                $gte: ISODate("1996-07-01T00:00:00Z"),
                $lt: ISODate("1998-06-01T00:00:00Z")
              }
            }
          },
          {
            $group: {
              _id: null,
              pedidos: {$sum: 1},
              com_envio: {$sum: {$cond: [{ $ne: ["$shipped_date", null]}, 1, 0]}},
              soma_dias: {
                $sum: {
                  $dateDiff: {startDate: "$order_date", endDate: "$shipped_date", unit: "day"}
                }
              },
              apos_requerida: {$sum: {$cond: [{ $gt: ["$shipped_date", "$required_date"]}, 1, 0]}}
            }
          }
        ],
        as: "res"
      }
    },
    {
      $set: {
        pedidos: {$ifNull: [{ $first: "$res.pedidos"}, 0]},
        com_envio: {$ifNull: [{ $first: "$res.com_envio"}, 0]},
        soma_dias: {$ifNull: [{ $first: "$res.soma_dias"}, 0]},
        apos_requerida: {$ifNull: [{ $first: "$res.apos_requerida"}, 0]}
      }
    },
    {
      $project: {
        _id: 0,
        shipper_id: "$_id",
        transportadora: "$company_name",
        pedidos: 1,
        com_envio: 1,
        sem_data: {$subtract: ["$pedidos", "$com_envio"]},
        soma_dias: 1,
        apos_requerida: 1,
        media_dias: {
          $cond: [
            { $eq: ["$com_envio", 0]},
            null,
            { $divide: [{ $toDecimal: "$soma_dias"}, "$com_envio"]}
          ]
        }
      }
    },
    { $sort: {shipper_id: 1}}
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.14 Par Q14/P14

PostgreSQL:

```
-- Q14 – Descontos sobre o valor dos pedidos
-- Pergunta: Quanto os descontos reduzem o valor bruto dos itens?
-- Resultado: Por categoria: valor bruto, desconto concedido, valor após descontos e taxa
ponderada de desconto.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
SELECT category_id,sum(bruto) bruto,sum(desconto) desconto,sum(valor) valor,
100*sum(desconto)/nullif(sum(bruto),0) percentual_desconto
```

```
FROM nw_analytics.vw_item_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01'
GROUP BY category_id ORDER BY category_id;
```

MongoDB:

```
// P14 – Descontos sobre o valor dos pedidos
// Pergunta: Quanto os descontos reduzem o valor bruto dos itens?
// Resultado: Por categoria: valor bruto, desconto concedido, valor após descontos e taxa
ponderada de desconto.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {$unwind: "$items"},
    {
      $set: {
        bruto: {$multiply: ["$items.unit_price", "$items.quantity"]},
        desconto: {$multiply: ["$items.unit_price", "$items.quantity", "$items.discount"]},
        valor: {
          $multiply: [
            "$items.unit_price",
            "$items.quantity",
            {$subtract: [Decimal128("1"), "$items.discount"]}
          ]
        }
      }
    },
    {
      $group: {
        _id: "$items.product.category._id",
        bruto: {$sum: "$bruto"},
        desconto: {$sum: "$desconto"},
        valor: {$sum: "$valor"}
      }
    },
    {
      $project: {
        _id: 0,
        category_id: "$_id",
        bruto: 1,
        desconto: 1,
        valor: 1,
        percentual_desconto: {
          $cond: [
            {$eq: ["$bruto", 0]},
            null,
            {$divide: [{$multiply: ["$desconto", Decimal128("100")]}, "$bruto"]}
          ]
        }
      }
    },
    {$sort: {category_id: 1}}
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.15 Par Q15/P15

PostgreSQL:

```
-- Q15 – Vendas próprias e da equipe
-- Pergunta: Quanto foi registrado por cada funcionário e sua equipe?
-- Resultado: Por funcionário: tamanho da equipe (ele e subordinados), valor próprio e valor da
equipe; equipes se sobrepõem.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH RECURSIVE equipe AS (
  SELECT employee_id gestor_id, employee_id membro_id, ARRAY[employee_id] caminho FROM nw.employees
  UNION ALL
  SELECT q.gestor_id, e.employee_id, q.caminho || e.employee_id
  FROM equipe q JOIN nw.employees e ON e.reports_to=q.membro_id
  WHERE NOT e.employee_id=ANY(q.caminho)
), vendas AS (
  SELECT employee_id, count(*) pedidos, sum(valor) valor
```

```

FROM nw_analytics.vw_pedido_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01' GROUP BY employee_id
)
SELECT q.gestor_id,count(*) membros,
coalesce(sum(v.valor) FILTER (WHERE q.membro_id=q.gestor_id),0) valor proprio,
coalesce(sum(v.pedidos),0) pedidos_equipe,coalesce(sum(v.valor),0) valor_equipe
FROM equipe q LEFT JOIN vendas v ON v.employee_id=q.membro_id
GROUP BY q.gestor_id ORDER BY q.gestor_id;

```

MongoDB:

```

// P15 – Vendas próprias e da equipe
// Pergunta: Quanto foi registrado por cada funcionário e sua equipe?
// Resultado: Por funcionário: tamanho da equipe (ele e subordinados), valor próprio e valor da
equipe; equipes se sobrepõem.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "employees",
  pipeline: [
    {
      $graphLookup: {
        from: "employees",
        startWith: "$_id",
        connectFromField: "_id",
        connectToField: "reports_to",
        as: "subordinados"
      }
    },
    {
      $set: {membro_ids: {$setUnion: [["_id"], "$subordinados._id"]}},
    },
    {
      $lookup: {
        from: "orders",
        localField: "membro_ids",
        foreignField: "employee_id",
        pipeline: [
          {
            $match: {
              order_date: {
                $gte: ISODate("1996-07-01T00:00:00Z"),
                $lt: ISODate("1998-06-01T00:00:00Z")
              }
            }
          },
          {
            $group: {
              _id: "$employee_id",
              pedidos: {$sum: 1},
              valor: {
                $sum: {
                  $sum: {
                    $map: {
                      input: "$items",
                      as: "i",
                      in: {
                        $multiply: [
                          "$$i.unit_price",
                          "$$i.quantity",
                          {$subtract: [Decimal128("1"), "$$i.discount"]}
                        ]
                      }
                    }
                  }
                }
              }
            }
          }
        ]
      }
    },
    {
      $project: {
        _id: 0,
        gestor_id: "$_id",
        membros: {$size: "$membro_ids"},
        valor proprio: {
          $sum: {
            $map: {
              input: {$filter: {input: "$vendas", as: "v", cond: {$eq: [ "$$v._id", "$_id" ]}}},
              as: "v",
              in: "$$v.valor"
            }
          }
        }
      }
    }
  ],
  as: "vendas"
}

```

```

    },
    pedidos_equipe: {$sum: "$vendas.pedidos"},
    valor_equipe: {$sum: "$vendas.valor"}
  }
},
{$sort: {gestor_id: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.16 Par Q16/P16

PostgreSQL:

```

-- Q16 – Distribuição trimestral por ano e categoria
-- Pergunta: Como o valor se distribui por ano, trimestre e categoria?
-- Resultado: Valor por ano e trimestre, total e por categoria, marcando trimestres com cobertura
parcial dos dados.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH t AS (
  SELECT extract(year FROM order_date)::int ano,
         extract(quarter FROM order_date)::int trimestre, category_id, valor
  FROM nw_analytics.vw_item_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01'
)
SELECT ano, trimestre, category_id, sum(valor) valor,
       CASE WHEN grouping(category_id)=1 THEN 'total' ELSE 'categoria' END nivel,
       (make_date(ano, (trimestre-1)*3+1, 1) < DATE '1996-07-04' OR
        (make_date(ano, (trimestre-1)*3+1, 1) + INTERVAL '3 months')::date > DATE '1998-05-07') parcial
FROM t GROUP BY GROUPING SETS ((ano, trimestre, category_id), (ano, trimestre))
ORDER BY ano, trimestre, category_id NULLS FIRST;

```

MongoDB:

```

// P16 – Distribuição trimestral por ano e categoria
// Pergunta: Como o valor se distribui por ano, trimestre e categoria?
// Resultado: Valor por ano e trimestre, total e por categoria, marcando trimestres com cobertura
parcial dos dados.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {$unwind: "$items"},
    {
      $set: {
        ano: {$year: "$order_date"},
        trimestre: {$toInt: {$ceil: {$divide: [{$month: "$order_date"}, 3]}}},
        valor: {
          $multiply: [
            "$items.unit_price",
            "$items.quantity",
            {$subtract: [Decimal128("1"), "$items.discount"]}
          ]
        }
      }
    }
  ],
  $facet: {
    total: [
      {$group: {_id: {ano: "$ano", trimestre: "$trimestre"}, valor: {$sum: "$valor"}}},
      {
        $project: {
          _id: 0,
          ano: "$_id.ano",
          trimestre: "$_id.trimestre",
          category_id: {$literal: null},
          valor: 1,
          nivel: {$literal: "total"}
        }
      }
    ],
    categoria: [
      {

```

```

        $group: {
          _id: {
            ano: "$ano",
            trimestre: "$trimestre",
            category_id: "$items.product.category._id"
          },
          valor: {$sum: "$valor"}
        },
      },
    {
      $project: {
        _id: 0,
        ano: "$_id.ano",
        trimestre: "$_id.trimestre",
        category_id: "$_id.category_id",
        valor: 1,
        nivel: {$literal: "categoria"}
      }
    }
  ]
},
{$project: {linhas: {$concatArrays: ["$total", "$categoria"]}}},
{$unwind: "$linhas"},
{$replaceWith: "$linhas"},
{
  $set: {
    inicio_trimestre: {
      $dateFromParts: {
        year: "$ano",
        month: {$add: [{$multiply: [{$subtract: ["$trimestre", 1]}, 3]}, 1]}
      }
    }
  }
},
{
  $set: {
    parcial: {
      $or: [
        {$lt: ["$inicio_trimestre", ISODate("1996-07-04T00:00:00Z")]},
        {
          $gt: [
            {$dateAdd: {startDate: "$inicio_trimestre", unit: "month", amount: 3}},
            ISODate("1998-05-07T00:00:00Z")
          ]
        }
      ]
    }
  }
},
{$unset: "inicio_trimestre"},
{$sort: {ano: 1, trimestre: 1, category_id: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

Apêndice B — Views, procedures e recursos documentais

00_views.sql

```
-- Camada da Entrega 3. Não altera as views históricas nem os dados de nw.
CREATE SCHEMA IF NOT EXISTS nw_analytics;

CREATE OR REPLACE VIEW nw_analytics.vw_item_valor AS
SELECT i.order_id, i.product_id, o.order_date, o.customer_id,
       o.employee_id, p.category_id, i.quantity,
       i.unit_price * i.quantity AS bruto,
       i.unit_price * i.quantity * i.discount AS desconto,
       i.unit_price * i.quantity * (1-i.discount) AS valor
FROM nw.order_items i
JOIN nw.orders o USING (order_id)
JOIN nw.products p USING (product_id);

CREATE OR REPLACE VIEW nw_analytics.vw_pedido_valor AS
SELECT o.order_id, o.order_date, o.customer_id, o.employee_id,
       o.shipper_id, o.required_date, o.shipped_date,
       count(i.product_id) AS itens,
       coalesce(sum(i.quantity),0) AS unidades,
       coalesce(sum(i.unit_price*i.quantity*(1-i.discount)),0) AS valor
FROM nw.orders o LEFT JOIN nw.order_items i USING (order_id)
GROUP BY o.order_id;

CREATE OR REPLACE VIEW nw_analytics.vw_vendas_mensais AS
SELECT date_trunc('month',order_date)::date AS mes,
       count(*) AS pedidos, sum(valor) AS valor,
       sum(valor)/count(*) AS ticket
FROM nw_analytics.vw_pedido_valor GROUP BY 1;

CREATE OR REPLACE VIEW nw_analytics.vw_envio_transportadora AS
SELECT s.shipper_id, s.company_name AS transportadora,
       count(o.order_id) AS pedidos,
       count(o.shipped_date) AS com_envio,
       count(o.order_id)-count(o.shipped_date) AS sem_data,
       coalesce(sum(o.shipped_date-o.order_date),0) AS soma_dias,
       avg(o.shipped_date-o.order_date) AS media_dias,
       count(*) FILTER (WHERE o.shipped_date>o.required_date) AS apos_requerida
FROM nw.shippers s LEFT JOIN nw.orders o USING (shipper_id)
GROUP BY s.shipper_id;

COMMENT ON VIEW nw_analytics.vw_item_valor IS
'Valor exato após descontos, sem frete e sem arredondamento por item.';
COMMENT ON VIEW nw_analytics.vw_pedido_valor IS
'Uma linha por pedido; pedidos sem itens aparecem com valor zero.';
COMMENT ON VIEW nw_analytics.vw_vendas_mensais IS
'Meses com movimento; a consulta temporal completa o calendário do recorte.';
COMMENT ON VIEW nw_analytics.vw_envio_transportadora IS
'Envio registrado não comprova entrega; data nula significa ausência de registro.';
```

10_procedures.sql

```
-- Relatórios tabulares: abrir transação, CALL, FETCH e COMMIT.
CREATE OR REPLACE PROCEDURE nw_analytics.sp_resumo_vendas_periodo(
    IN inicio date, IN fim date, INOUT resultado refcursor)
LANGUAGE plpgsql AS $$
BEGIN
    IF inicio IS NULL OR fim IS NULL OR inicio>=fim THEN
        RAISE EXCEPTION 'Informe início anterior ao fim exclusivo';
    END IF;
    OPEN resultado FOR
    SELECT i.category_id,c.category_name categoria,
           count(DISTINCT i.order_id) pedidos,sum(i.valor) valor,
           sum(i.valor)/count(DISTINCT i.order_id) ticket_categoria
    FROM nw_analytics.vw_item_valor i
    JOIN nw.categories c USING(category_id)
    WHERE i.order_date>=inicio AND i.order_date<fim
    GROUP BY i.category_id,c.category_name ORDER BY i.category_id;
END;
$$;

CREATE OR REPLACE PROCEDURE nw_analytics.sp_clientes_inativos(
    IN referencia date, IN meses integer, INOUT resultado refcursor)
LANGUAGE plpgsql AS $$
BEGIN
    IF referencia IS NULL OR meses IS NULL OR meses<1 THEN
        RAISE EXCEPTION 'Informe referência e meses positivos';
    END IF;
```

```

END IF;
OPEN resultado FOR
WITH historico AS (
    SELECT c.customer_id,c.company_name cliente,max(o.order_date) ultima
    FROM nw.customers c LEFT JOIN nw.orders o
    ON o.customer_id=c.customer_id AND o.order_date<=referencia
    GROUP BY c.customer_id
)
SELECT *,referencia-ultima recencia,
    CASE WHEN ultima IS NULL THEN 'sem compras' ELSE 'inativo' END situacao
FROM historico
WHERE ultima IS NULL OR ultima<(referencia-make_interval(months=>meses))::date
ORDER BY customer_id;
END;
$$;

CREATE OR REPLACE PROCEDURE nw_analytics.sp_desempenho_equipe(
    IN gestor integer, IN inicio date, IN fim date, INOUT resultado refcursor)
LANGUAGE plpgsql AS $$
BEGIN
    IF inicio IS NULL OR fim IS NULL OR inicio>=fim THEN
        RAISE EXCEPTION 'Informe início anterior ao fim exclusivo';
    END IF;
    IF gestor IS NULL OR NOT EXISTS (
        SELECT 1 FROM nw.employees WHERE employee_id=gestor
    ) THEN RAISE EXCEPTION 'Gestor inexistente'; END IF;
    OPEN resultado FOR
    WITH RECURSIVE equipe AS (
        SELECT employee_id,ARRAY[employee_id] caminho
        FROM nw.employees WHERE employee_id=gestor
        UNION ALL
        SELECT e.employee_id,q.caminho||e.employee_id
        FROM equipe q JOIN nw.employees e ON e.reports_to=q.employee_id
        WHERE NOT e.employee_id=ANY(q.caminho)
    ), vendas AS (
        SELECT employee_id,count(*) pedidos,sum(valor) valor
        FROM nw_analytics.vw_pedido_valor
        WHERE order_date>=inicio AND order_date<fim GROUP BY employee_id
    )
    SELECT gestor gestor_id,count(*) membros,
        coalesce(sum(v.valor) FILTER (WHERE q.employee_id=gestor),0) valor_proprio,
        coalesce(sum(v.pedidos),0) pedidos_equipe,
        coalesce(sum(v.valor),0) valor_equipe
    FROM equipe q LEFT JOIN vendas v USING(employee_id);
END;
$$;

```

11_exemplos_procedures.sql

```

BEGIN;
CALL nw_analytics.sp_resumo_vendas_periodo(
    '1996-07-01','1998-06-01','vendas');
FETCH ALL FROM vendas;
CALL nw_analytics.sp_clientes_inativos('1998-05-06',6,'inativos');
FETCH ALL FROM inativos;
CALL nw_analytics.sp_desempenho_equipe(2,'1996-07-01','1998-06-01','equipe');
FETCH ALL FROM equipe;
COMMIT;

```

recursos_documentais.js

```

var b = db.getSiblingDB('northwind');
// As três condições precisam ocorrer no MESMO elemento do array.
var mesmoItem = [
    { $match: { items: { $elemMatch: {
        'product._id': 11, quantity: { $gte: 10 }, discount: { $gt: Decimal128('0') }
    } } } },
    { $project: { _id: 1, items: { $filter: { input: '$items', as: 'i', cond: { $and: [
        { $eq: [ '$$i.product._id', 11 ] }, { $gte: [ '$$i.quantity', 10 ] },
        { $gt: [ '$$i.discount', Decimal128('0') ] }
    ] } } } } },
    { $sort: { _id: 1 } }
];
// Contraste: as mesmas condições escritas separadamente podem ser atendidas por itens
diferentes.
var independentes = { 'items.product._id': 11, 'items.quantity': { $gte: 10 },
    'items.discount': { $gt: Decimal128('0') } };
// Um documento de entrada continua sendo um documento de saída.
var pedido = [
    { $match: { _id: 10248 } },
    { $project: { _id: 1, produtos: { $map: { input: '$items', as: 'i', in: '$$i.product.name' } } },

```

```

    valor:{$reduce:{input:'$items',initialValue:Decimal128('0'),in:{$add:[
      '$$value',{multiply:[ '$$this.unit_price','$$this.quantity',
        {$subtract:[Decimal128('1'),'$$this.discount']}]}]}]}]}
  ];
// Facetas diferentes sobre os mesmos documentos de entrada.
var painel = [
  {$match:{order_date:{$gte:ISODate('1997-01-01'),$lt:ISODate('1998-01-01')}}},
  {$facet:{
    volume:[{$count:'pedidos'}],
    envio:[{$group:{_id:{$cond:[{$eq:['$shipped_date',null]],'sem data','com data'}},
      pedidos:{$sum:1}},{$sort:{_id:1}}],
    tamanho:[{$group:{_id:{$size:'$items'},pedidos:{$sum:1}},{$sort:{_id:1}}]
  }}
];
print(EJSON.stringify({mesmo_item:b.orders.aggregate(mesmoItem).toArray(),
  condicoes_independentes:b.orders.countDocuments(independentes),

pedido:b.orders.aggregate(pedido).toArray(),painel:b.orders.aggregate(painel).toArray()},null,2))
;

```

mapreduce.js

```

// MapReduce é requisito didático legado. Somente quantidades inteiras.
var b = db.getSiblingDB('northwind');
var resultado = b.runCommand({
  mapReduce: 'orders',
  map: function () {
    this.items.forEach(function (i) { emit(i.product._id, i.quantity); });
  },
  reduce: function (produto, quantidades) { return Array.sum(quantidades); },
  out: { inline: 1 }
});
if (resultado.ok !== 1) throw new Error(EJSON.stringify(resultado));
var pipeline = [
  {$unwind: '$items'},
  {$group: {_id: '$items.product._id', value: {$sum: '$items.quantity'}}},
  {$sort: {_id: 1}}
];
var moderno = b.orders.aggregate(pipeline).toArray();
var legado = resultado.results.sort((a,b) => a._id-b._id);
require('assert').deepStrictEqual(
  legado.map(x=>[Number(x._id),Number(x.value)]),
  moderno.map(x=>[Number(x._id),Number(x.value)])
);
print(EJSON.stringify({equivalente:true,produtos:moderno.length,
  unidades:moderno.reduce((s,x)=>s+Number(x.value),0),resultados:legado},null,2));

```